一个NP-Complete卡牌游戏在特定规模下的最优策略启发式求解

概述

我独立设计并实现了一款,规则简单,需要运气和策略的 Unity 引擎开发的回合制PVE卡牌游戏，电脑的策略基于一个部署了训练好的AI集成模型的本地服务器。

我将游戏规则抽象为基于特定规则的子序列和得分最大化问题,并深入研究它后,发现了实际的决策空间随着操作步数呈组合爆炸式增长.为了从根本上理解问题的计算复杂性，我分析了该卡牌游戏的泛化版本。通过将其规约为子集和问题，证明了该问题是NP-Hard而且是NP-Complete的。这一结论深刻地揭示了，任何试图寻找通用精确解的算法（如穷举搜索）都将面临不可避免的扩展性瓶颈。正因如此，当问题规模稍大时，精确解在计算上变得不可行。这为项目的核心策略——即放弃追求通用精确解的算法，转而研究在特定规模下，使用AI代理模型来高效逼近最优解——提供了坚实的理论依据。

面对这种精确解存在但计算成本过高的困境，本文探索了一种基于 Transformer 模型与遗传算法集成组成的轻量化代理模型（Surrogate Model）的求解通用方法尝试。其核心思想是：利用精确穷举算法在一个可计算的规模内生成高质量的训练样本，然后训练一个计算成本低廉的AI模型来学习其决策行为。

本文的核心工作在于

1.通过严谨的实验证明了在特定规模下,NP-Hard问题的穷举递归算法的最优策略能被构建的集成模型能够以高精度复现,并进行了一些泛化的尝试.这验证了当面临这类动态的、决策空间巨大的NP-Complete问题时，AI代理模型作为一种计算成本可控的替代方案的潜力。

2.设计并开发了一款完整的游戏项目，涵盖了从客户端到服务器端(服务器策略基于AI代理模型)的全栈实现

相对于单纯的学术文章而言 本文也是一个高完成度项目的报告，,在算法,AI,工程上都做了一些创新,并达到了一定深度

项目亮点

- **问题建模的抽象与实现**:
- **严谨的算法分析**
- **自定义问题的NP-complete性质证明以及相关讨论 **

> 注:基本证明处于正文,相关讨论在附录中

- **全面的ai模型评估体系**
- **代理模型的提出和实现**
- **集成模型的批处理和多线程优化**:
- **较复杂的游戏框架的自定义和扩展**
- **端到端的系统构建与实现**

- **讨论了一种能够很好表达组合问题的树结构(附录)**

> 注 代码具体实现在code_path中

文章结构

本文分为4个主要部分:

1.游戏规则以及算法分析

本部分将详细介绍游戏规则以及编程实现，包括：

- **游戏规则**
- **算法分析：**
- **NP-Complete的证明**
- **算法优化讨论与AI代理模型的提出：**

2. AI 技术细节

本部分将详细介绍游戏中 AI 系统的工作原理以及核心函数实现，包括：

- **AI模型的实现：**
- **模型训练**
- **数据预处理的消融实验讨论**
- **模型评测**
- **集成模型的评测与优化**
- **Transformer的建模范式冲突与归纳偏置分析讨论**

3. 服务器的搭建与游戏架构

本部分将展示 Unity客户端的架构与游戏实机演示

- **AI模型服务器的搭建**
- **游戏的客户端架构：**
- **游戏实机演示：**

4.结语

第一部分 游戏规则以及算法分析

游戏规则以及玩法示例

游戏规则与“连连看”有点类似，玩家需要通过配对卡牌来消除它们得分，到电脑的回合时,电脑会获取当前游戏的局面,将动态的卡牌局面数据传输到本地AI服务器,得到ai预测的最佳行动后,在客户端自动执行并向用户展示,在3回合后得分高者获胜。

开局

1. 系统创建一个包含52个数字的列表 P ，每个数字从1到13，每个数字严格重复4次（共 13 * 4 = 52 个元素）。
2. 从列表 P 中**随机抽取 6 个不同的元素**，得到列表 A 。
    - **示例：** 假设我们得到 A = {1, 2, 3, 7, 4, 6 } 。
    - **(补充说明)** 本文具体研究的具体特例的 A 的元素个数固定是6。
3. 再从剩余列表 P 中**随机抽取 3 个允许重复的元素**，得到列表 B 。
    - **示例：** 假设我们得到 B = {4, 3, 9 } 。
    - **(补充说明)** 本文具体研究的具体特例的 B 的元素个数固定是3。

可插入位置

- 列表 A 中的元素之间及其两端是允许插入的位置。
- 对于示例 A = {1, 2, 3, 7, 4, 6 } ，可插入位置用序号表示是： ① 1 ② 2 ③ 3 ④ 7 ⑤ 4 ⑥ 6 ⑦
- 总共有 6 + 1 = 7 个位置。

回合开始

- 玩家允许在 B 中选取一张牌插入 A 中的一个可插入位置。
- 牌被选取并插入后，会从 B 中移除，并添加到 A 中。

示例回合流程

第一次出牌

1. 玩家选择将牌 3 从 B 中取出，插入到 A 中的位置 ① (即 A[0] 之前的位置)。
2. A, B 被更新： * A 变为 {3, 1, 2, 3, 7, 4, 6 } * B 变为 {4, 9 }

匹配规则

- 观察新插入的牌 (3)。在更新后的 A 中，查找数值上与新插入的牌相同的元素。
- **示例：** 新插入的牌是 3 。在 {3, 1, 2, 3, 7, 4, 6 } 中，A[3] 的值也是 3 ，与新加入的 3 相匹配。
- **(补充说明)** 在 A 中最多只会有一个元素与新加入的牌匹配，不会出现两个匹配。这是因为初始化时， A 中的元素都是独一无二的。

得分与移除 (第一次出牌)

1. 找到匹配后，形成一个序列：从新插入的牌开始，到匹配到的元素结束（包含这两个元素）。
    - **示例：** 新插入的 3 在 A[0] ，匹配到的 3 在 A[3] 。序列是 A[0] 到 A[3] ，即 {3, 1, 2, 3 } 。
2. 这个匹配的序列将从 A 中被移除。
3. 记录得分：得分为序列中所有元素的数值之和。
    - **示例：** 得分1 = 3 + 1 + 2 + 3 = 10 。
4. A 会动态更新： * A 移除 {3, 1, 2, 3 } 后，变为 {7, 4, 6 } 。* B 没有改变，仍为 {4, 9 } 。
5. 可插入位置随之改变：对于 A = {7, 4, 6 } ，可插入位置变成 ① 7 ② 4 ③ 6 ④

第二次出牌

1. 玩家选择将牌 9 从 B 中取出，插入到 A 中的位置 ④
2. A, B 被更新： * A 变为 {7, 4, 6, 9 } 。* B 变为 {4 } 。
3. 检查匹配：新插入的牌是 9 。在 {7, 4, 6, 9 } 中，没有其他元素数值为 9。
4. 没有匹配发生，所以得分2为 0。
5. A 和 B 保持当前状态。

6. 可插入位置变成 ① 7 ② 4 ③ 6 ④ 9 ⑤

第三次出牌

1. 玩家选择将最后一张牌 `4` 从 B 中取出，插入到 A 中的位置 ⑤
2. A, B 被更新： * A 变为 {7, 4, 6, 9, 4 } 。* B 变为 `{}` (空)。
3. 检查匹配：新插入的牌是 `4`。在 {7, 4, 6, 9, 4 } 中，`A[1]` 的数值也是 `4`，与 `A[4]` 相匹配。
4. 匹配到的序列是 A[1] 到 A[4] ，即 {4, 6, 9, 4 } 。
5. 记录得分：得分3 = 4 + 6 + 9 + 4 = 23 。
6. A 移除 {4, 6, 9, 4 } 后，变为 {7 } 。
7. B 已经是空列表 `{}`。

回合结束

- 所有玩家（在此示例中是玩家自己）已经出了3次牌， B 中的所有元素都被插入到 A 中。
- 回合结束。

总得分

- 总得分为该回合所有得分之和。
- **示例：** 总得分 = text{得分1} + text{得分2} + text{得分3} = 10 + 0 + 23 = 33 。

穷举递归算法分析

我们要计算的是给定A,B后的最佳策略

给定的A,B: A是一个长度为m,元素间不可重复的整数列表, 范围是1~13 B是一个长度为n,元素间可重复的整数列表, 范围是1~13

策略: 这里的这个一个 策略 指的是 B中的元素以什么样的顺序插入到A中的哪些位置(也就是说，策略包括两个部分：B 的插入顺序，以及插入到 A 中的位置。)。

匹配：当你从列表 B 中选择一张牌，并将其插入到列表 A 的某个指定位置之后，立即检查：

① 新插入的这张牌的数值，是否与当前列表 A 中已经存在的某一张牌的数值完全相同。 ② 如果存在这样一张数值相同的、原本就在 A 中的牌，那么就称之为发生了一次“匹配”。

匹配序列 与 ***移除***： ：

发生匹配时，会确定一个“匹配序列”。这个序列包含 ①新插入的那张牌。 ②原本就在 A 中且数值相同的那张牌。 ③位于这两张牌在列表 A 中位置之间的所有牌。 这个完整的序列将从列表 A 中被移除。

得分： 这次操作的得分，等于被移除的“匹配序列”中所有牌的数值之和。

没有发生匹配和得分为0： 如果新插入的牌的数值，在当前列表 A 中找不到任何一张数值相同的、原本就存在的牌，那么没有发生“匹配",此时列表 A 的匹配序列为空,不会移除任何元素，这次操作的得分为 0。

总得分: A,B使用此策略后会将B中的元素依次插入到A中 每次插入会产生一个 匹配 的判断 ,产生相应的得分,然后计算3次匹配对应的得分的和,也就是总得分

最佳策略: 指的是 能使得总得分最大的策略

首先进行一些数学估计:

时间复杂度分析(不考虑匹配):

对于 B 中的每一个元素，它尝试在当前的 A 的所有可能位置插入，然后递归处理剩余的元素。假设 length(B) = n， length(A) = m，那么在第一步有 n 种元素选择，每种元素有 m+1 个插入位置，总共有 n * (m+1) 种可能。第二步B有 n-1 个元素，插入位置变为 m+2 个，依此类推得到n!*(m+1)*(m+2)...(m+n)=n!·(m+n)!/m!。所以得到大致的复杂度是

$$O\left(n! \cdot \frac{(m+n)!}{m!}\right)$$

这个时间复杂度非常高。

(但由于每次插入后A的长度的改变并不确定, 可能没有匹配导致的长度加一 也可能产生了匹配,移除了2~length(A)个元素 所以这个估计并不是精确的)

基于原问题的特定规模展开研究

接着我们对原问题的一个特定规模(A 长度为6,B 的长度为3) 进行研究,注意全文的研究都是基于此展开的

问题建模

首先我们解决能处理匹配过程并计分的函数 insertion (), 由于A会随着插入以及匹配而改变 所以在调用这个函数时,这里的输入的A的长度不再是初始化的6 而是0~8(因为当A是0~8时才有可能去调用insertion,A不可能小于0且A等于9时 B=[],B不能再选取元素插入),在插入之后,由于匹配,A的长度会变为0~9

定义insertion(A, b, pos)函数 (实现见code_path)

它将元素b插入到列表A的指定位置pos，并处理可能发生的匹配情况。

返回 [得分, 由于匹配移出的长度, 新生成的A的长度, 是否匹配(0或1), 新生的A]

这个问题的关键问题之一在于如何设计数据结构来表示插入策略。

我使用的策略结构是一个形如 3×2 的二维矩阵（或列表），我们称它为 move。这个矩阵的每一行都代表一个插入操作，共两列：

> 第一列表示元素在数组 `B` 中的索引；
>
> 第二列表示将该元素插入到当前数组 `A` 中的哪个位置。

需要注意的是，这里的“插入位置”是相对于最新的 A 状态而言的，而不是初始的 A

举个例子：

B = [1, 2, 3] move = [[1, 6], [0, 3], [2, 0]]

这个策略表示：

> 第一步：将 `B[1]`（即元素 `2`）插入到当前 `A` 的索引位置 `6`；

第二步：将 `B[0]`（即元素 `1`）插入到更新后的 `A` 的索引位置 `3`；

第三步：将 `B[2]`（即元素 `3`）插入到再次更新后的 `A` 的索引位置 `0`。

每一步都在当前版本的 A 上进行操作，每次B[i]的插入都会导致A会由于插入本身和插入后的匹配而更新

所以可以设计递归算法函数:

Recursive_StrategyAndScore(A, B, original_indices=None, move_sequence=None, used_indices=None)(实现见code_path)

返回 理论最大得分,理论最大得分对应的操作

结合具体规模分析穷举的数据量

计算穷举当A,B没有给定且不发生任何匹配消除时,所有的将B插入A后的所有可能性(注意这里仅仅是"最终静态组合数")

如果仅考虑将B(长度为3的数组)中的牌插入A(长度为6的数组)里,不考虑A,B上有点数,求解有多少中插入的可能,那么这可以看作是一个多重集排列问题:从{A,A, A,A, A,A, B,B,B}构成的集合中不放回的拿出元素,全部放到一个一维数组里,能构成多少种长度为9的不同的数组?
所以是

$$\binom{9}{6} \cdot \binom{3}{3} = \binom{9}{6}$$

然后由于现在的数组的牌只有A,B的区别,位置已经定下来,但是 还没有点数 现在给他们加上点数

每个A的值是1~13,且不能重复 所以给A加上点数的所有可能方式:P(13,6)
每个B的值也是1~13,但是可以重复 给B加上点数的所有可能方式:13^3

所以这是所有的可能插入得到的最终可能

$$\binom{9}{3} \cdot \frac{13!}{(13-6)!} \cdot 13^3 = 228{,}012{,}744{,}960$$

假设每个样本占10B 数据超过2000GB 这是一个非常大的数据量 (这是一个估计,就如前面所说,每次插入可能导致A数组变动,从而导致之后的决策发生改变,而且不同顺序的插入是不一样的

匹配会导致B的插入顺序是需要考虑的 比如 b1插入1号位,b2 插入2号位,这样的顺序反过来 ,b2 插入2号位,b1插入1号位,由于匹配规则,会产生不同的结果 所以是两种情况

这里只考虑了不发生匹配的情况,这也同时导致了有些插入可能是不合法的 而且匹配会同时导致A的长度改变,我们无法直接判断匹配带来的是更多还是更少的情况)

基于蒙特卡洛模拟得到更好的数据量估计

为了估计真实的匹配造成的影响,这里使用了基于概率的统计数据分析(蒙特卡洛模拟) 来研究一这特例 : 对于当 length(A)=6 length(B)=3 时

随机生成A,B length(A)=6 length(B)=3 随机不放回的抽取B中的一张牌,随机插入到A中,重复2次调用simulate_insertion_tool直到B中没有元素,第二次调用是基于上一次改变后的A,B,

```
{
        A, B = get_A_and_B_random()

        card = random.choice(B)

        pos=random.randint(0,6)

        _, _, new_length1, _, newA = simulate_insertion_tool(A, card, pos)

        all_lengths1.append(new_length1)

#第一次改变后的A的长度期望
expected_value = sum(all_lengths1) /total_times

        B.remove(card)
        card = random.choice(B)
        pos = random.randint(0, len(newA))

        _, _, new_length2, _, newA = simulate_insertion_tool(newA, card, pos)

        all_lengths2.append(new_length2)

#第二次改变后的A的长度期望
expected_value = sum(all_lengths2) / total_times

}
```

获取函数的第三个返回值 以上过程重复20M次(total_times=20M) 进行统计, 编写代码运行 以下是我的结果:

总共执行了 20M 次 simulate_insertion_tool 调用。

```
 length(A1) 分布情况:
   length(A1)= 0: 559058  次, 占比: 2.80%
   length(A1)= 1: 1112357 次, 占比: 5.56%
   length(A1)= 2: 1669068 次, 占比: 8.35%
   length(A1)= 3: 2216588 次, 占比: 11.08%
   length(A1)= 4: 2775405 次, 占比: 13.88%
   length(A1)= 5: 3333920 次, 占比: 16.67%
   length(A1)= 7: 8333604 次, 占比: 41.67%

length(A1) 的期望值: 4.8603≈5

 length(A2)分布情况:
   length(A2)=0: 689171  次, 占比: 3.45%
   length(A2)=1: 1746955 次, 占比: 8.73%
   length(A2)=2: 2511990 次, 占比: 12.56%
   length(A2)=3: 3001901 次, 占比: 15.01%
   length(A2)=4: 3202618 次, 占比: 16.01%
   length(A2)=5: 3129061 次, 占比: 15.65%
   length(A2)=6: 3434722 次, 占比: 17.17%
```

```
    length(A2)=8: 2283582 次, 占比: 11.42%

length(A2) 的期望值: 4.1555≈4
```

simulate_insertion_tool在每个循环中只调用两次是因为第一次插入可能固定为length(A)+1=7 并且不用考虑第三次插入后的A的变化,因为此时length(B)等于0,在这之后算法已经停止了

第一次插入有 length(A)+1 个位置，第二次插入有 E[length(A1)]+1 个位置，第三次插入则应基于第二次插入后A的期望长度 E[length(A2)]+1 个位置

基于这些期望 得到估计 决策路径数量为

$$((\text{length}(A)+1)\cdot(E[\text{length}(A_1)]+1)\cdot(E[\text{length}(A_2)]+1))\cdot P(13,6)\cdot 13^3$$

(7×6×5)/ C(9,3)=2.5 这个结果变成不考虑匹配规则"最终静态组合数"的2.5倍 需要再次强调的是 这里估计的不是A与B的"最终静态组合数",而是决策的数量 因为这个模拟估计的并不仅仅是单纯的卡牌组合的数据量,而且考虑了匹配规则,也就是考虑了B的插入顺序,它是"动态组合"的个数 .因此实际的出牌策略比之前讨论的不考虑匹配的"最终静态组合数"还要大

时间复杂度的通过这个思想也能得到更精确的估计,但由于改变的是数据的规模 , 所以依然是阶乘的时间复杂度

穷举递归搜索（Recursive_StrategyAndScore）为了保证找到最优解而去尝试所有可能，时间复杂度是极高的。而strategy_TrueScore函数则可以快速验证任意一个给定策略的好坏。

它是NP的 而且有可能是NP-hard的

在Travelling salesman problem中，访问一个城市的决定会影响到下一个城市的距离和路径选择。本问题同样的，一次"匹配"操作会彻底改变"地图"（即列表A），影响后续所有"路径"的得分(即最佳出牌的决策)。这样结构的相似性使得问题很可能是NP-Hard的。 我们使用turing-reduction 来证明

NP-hard的证明

证明：卡牌游戏优化问题 (CARD_GAME_OPT) 是 NP-hard 的

1. 证明策略

为证明优化问题 `CARD_GAME_OPT` 是 NP-hard 的，我们将采用**图灵规约 (Turing Reduction)** 的方法，从一个已知的 NP-complete 问题——**子集和问题 (Subset Sum Problem)**——进行规约。

证明的核心逻辑遵循**反证法：** 首先，我们假设 `CARD_GAME_OPT` **不是** NP-hard 的，即存在一个能解决它的多项式时间算法。然后，我们将展示如何利用这个假想的算法作为预言机（Oracle），来设计**另一个**多项式时间算法，从而解决子集和问题。

如果以上逻辑成立，则意味着一个 NP-complete 问题可以在多项式时间内被解决，这将导出 P=NP 的结论。这与计算复杂性理论中 P≠NP 的基本假设相矛盾，因此我们最初的假设必然是错误的。

2. 相关问题定义

- **子集和问题 (Subset Sum Problem)**

    - **输入**：一个由 n 个**互异的**（distinct）正整数构成的**集合** $S = \{s_1, s_2, ..., s_n\}$ 和一个目标整数 T。
    - **问题**：是否存在 S 的一个子集 $S' \subseteq S$，使得 S' 中所有元素的和恰好等于 T？
    - **复杂度**：此问题是经典的 NP-complete 问题[1]。我们特指其处理互异整数集合的范式，以确保规约的有效性。

- **卡牌游戏优化问题 (CARD_GAME_OPT)**

    - **输入**：两个卡牌列表 A 和 B。
    - **问题**：找到一个出牌策略，使得最终获得的总得分**最大化**，并返回这个最大分值。

3. 规约证明

第一步：反证假设

假设 $CARD_GAME_OPT$ 存在一个多项式时间解法，我们称这个假想的算法为 $Solve_OPT$。我们可以将其视为一个预言机 O，它能在多项式时间内完成 $O(A, B) = max_score$ 的计算。

第二步：设计用于解决子集和问题的新算法

我们设计一个新算法，命名为 $SubsetSumSolver$，它以一个子集和问题的实例 (S, T) 作为输入。此算法通过调用预言机 O 来判断问题的解是否存在。

A. 预言机能力封装

$SubsetSumSolver$ 算法需要将其想要查询的数学问题"翻译"成预言机 O 能理解的游戏问题。为此，算法内部实现了一个核心函数 $MaxSubsetSum(S_sub)$。

- **函数 $MaxSubsetSum(S_sub)$ 的操作流程：**

    1. 该函数接收 S_sub 作为参数,$S_sub \subset S$。
    2. 根据该参数，函数程序化可以地构造*一个游戏实例 ☑ + S_sub)。其中 X 是一个预先选定的、大于 Sum(S) 的足够大的整数。

    > *构造方法比如:对于任意的S,我们都可以通过计算 S 中所有元素的和并加1来得到X，即 X = Sum(S) + 1. 然后构造 A=[X] 和 B=[X] + S_sub 这样的构造符合卡牌游戏的规定,并且对于任意的S显然成立

    3. 函数将构造出的实例 I 传递给预言机 O，并获取返回值 max_score。
    4. 函数计算并返回 $max_score - 2X$。

- **函数能力分析：** 由于 S 中元素互异且 X 值极大，得分的唯一途径是匹配两张 X 牌。因此,由于S_sub互异,$max_score - 2X$ 的结果恰好是输入集合 S_sub 的最大子集和（即 S_sub 中所有元素之和）。由于构造实例、调用预言机 O 和减法运算都是多项式时间的，所以 $MaxSubsetSum$ 函数本身也是多项式时间的。

B. $SubsetSumSolver$ 算法主体

该算法利用其内部函数 $MaxSubsetSum$ 来判断是否存在和为 T 的子集。

- **算法执行流程 $SubsetSumSolver(S, T)$：**

1. **初始判断**：若 `T < 0` 或 `T > MaxSubsetSum(S)`，则解显然不存在，算法返回“否”。
2. **构造性求解：** a. 初始化目标和 `T_rem = T`。b. 创建 `S` 的一个副本 `S_rem` 作为待考察元素的集合，并按从大到小的顺序排列。 c. 遍历 `S` 中的每一个元素 s_i ： i. 从 `S_rem` 中移除 s_i。ii. 判断是否应该将 s_i 加入最终的解： * 如果 s_i `<= T_rem`， 且 `T_rem - `s_i `<= MaxSubsetSum(S_rem)` (如果将s_i加入解的集合，那么代表在数字上无法凑齐T)

   > 这意味着如果选取 `$s_i$`，剩余的目标和 `T_rem - `s_i` **有可能**被剩下的元素 `S_rem` 凑齐。算法做出决策，将 `s_i` 视为解的一部分，并更新 `T_rem = T_rem - `$s_i$`。
3. **返回结果**：在遍历完所有元素后，如果 `T_rem` 恰好等于 `0`，则意味着算法成功地找到了一组元素 `S_solution`的组合，其和为 `T`。算法返回“是”。否则，返回“否”。

该算法的完全正确性 (Total Correctness) 包含：

```
A. 终止性 (Termination)：算法总能在有限步骤内停机。
B. 部分正确性 (Partial Correctness)：算法停机后的结果是正确的。
    而要证明部分正确性，又需要证明：
        B1. 可靠性 (Soundness)：算法返回“是”时，问题确实有解。
        B2. 完备性 (Completeness)：问题有解时，算法一定会返回“是”。
```

终止性显然成立,因为算法遍历S后就会终止

****证明：可靠性 (Soundness)**

问题：如果`SubsetSumSolver`算法结束后T_rem = 0，为什么说就代表了算法成功地找到了一组元素 `S_solution`的组合？

1. 定义循环不变量

让我们来定义一个在算法的循环过程中始终成立的等式。

设 `T_rem(k)` 为算法在考察完前 `k` 个元素后的 `T_rem` 值。 设 `S_solution(k)` 为算法在考察完前 `k` 个元素后的 `S_solution` 集合。

我们要证明的**循环不变量**是：

$$T_{rem}(k) + \sum_{s \in S_{solution}(k)} s = T$$

T_rem的行为是初始化为T,然后不断的减去有可能的s_i,也就是 T-已选入解集的元素之和

所以这个等式表明，在循环的每一步之后， **总目标T-已选入解集的元素之和** 加上 **已选入解集的元素之和**，结果永远等于**最初的总目标** `T`,等式本身是显然成立的。

2. 数学归纳法证明

我们将通过数学归纳法来证明这个不变量在整个算法执行期间始终成立。

- **基准情况 (Base Case):** `k = 0` 在循环开始之前（即考察了0个元素时）：
  - `S_solution(0)` 是空集 `[]`，所以其元素之和 `Sum(S_solution(0)) = 0`。
  - `T_rem(0)` 被初始化为 `T`。
  - 此时，不变量等式为：`T_rem(0) + Sum(S_solution(0)) = T + 0 = T`。
  - **结论**：在循环开始前，不变量成立。
- **归纳步骤 (Inductive Step)** 假设在考察完第 `k-1` 个元素后，不变量成立。即：

  $$T_{rem}(k-1) + \sum_{s \in S_{solution}(k-1)} s = T$$

  现在，我们来分析算法在考察第 `k` 个元素 s_k 时的两种情况：
  - **情况一：解集 `S_solution` 加入 s_k 之后** 根据算法的决策逻辑，这会发生。此时，状态更新如下：
    - `S_solution(k)` = `S_solution(k-1)` ∪ {s_k}，因此 `Sum(S_solution(k)) = Sum(S_solution(k-1)) + s_k`。
    - `T_rem(k)` = `T_rem(k-1) - s_k`。

    我们将新状态代入不变量等式的左侧：`T_rem(k) + Sum(S_solution(k)) = (T_rem(k-1) - s_k) + (Sum(S_solution(k-1)) + s_k) = T_rem(k-1) + Sum(S_solution(k-1))` 根据我们的归纳假设，上式的结果等于 `T`。**因此，在这种情况下，不变量在第 `k` 步之后依然成立。**
  - **情况二：s_k 未被加入解集 `S_solution`** 在这种情况下，状态更新如下：
    - `S_solution(k)` = `S_solution(k-1)`，因此 `Sum(S_solution(k)) = Sum(S_solution(k-1))`。
    - `T_rem(k)` = `T_rem(k-1)`。

    我们将新状态代入不变量等式的左侧：`T_rem(k) + Sum(S_solution(k)) = T_rem(k-1) + Sum(S_solution(k-1))` 根据我们的归纳假设，上式的结果等于 `T`。**因此，在这种情况下，不变量在第 `k` 步之后也依然成立。**
- **结论** 通过数学归纳法，我们证明了这个循环不变量 `T_rem(k) + Sum(S_solution(k)) = T` 在算法的每一次迭代后都始终保持成立。

3. 推论

当算法遍历完所有 `n` 个元素并终止时，该不变量依然成立。设 `T_rem_final` 和 `S_solution_final` 为最终状态。我们有：

$$T_{rem_final} + \sum_{s \in S_{solution_final}} s = T$$

此时，算法的返回逻辑是检查 `T_rem_final` 是否等于 `0`。如果`T_rem_final = 0`，我们可以将它代入上述已被证明的等式中：

$$0 + \sum_{s \in S_{solution_final}} s = T$$

$$\sum_{s \in S_{solution_final}} s = T$$

证明完毕。 这表明，当且仅当算法终止时 `T_rem` 为0，我们构造的解集 `S_solution` 的元素之和才精确地等于原始目标 `T`。

因此算法结束时如果 T_rem 变成了 0，我们就能确定 Sum(S_solution) 一定等于 T

证明:完备性 (Completeness)

问题：如果问题本身真的存在一个解，那么算法结束时 T_rem 为什么必然会等于 0?

待证命题：对于任何一个确实存在解的子集和问题实例 `(S, T)`，报告中描述的 `SubsetSumSolver` 算法保证会终止，并返回"是"（即最终 `T_rem = 0`）。

采用反证法证明：

1. **做出反证假设** 假设待证命题为假。即，**存在**一个最优解 `S*`（其所有元素之和为 `T`），但 `SubsetSumSolver` 算法在执行后**失败**了，最终返回"否"（即算法终止时 `T_rem > 0`）。
2. **分析失败的必然原因** 如果算法最终结果不正确，那么在其按降序遍历元素的执行过程中 ，必然至少有一次做出了"错误"的决策。我们可以定位到算法**第一次**犯错的时刻。这个错误必然是：
   - 算法**没有选择**一个本应属于最优解 `S*` 的元素。我们称这个被错误"跳过"的元素为 `s*`。

   在考虑 `s*` 之前，所有比 `s*` 大的元素都已被正确处理。
3. **分析算法做出错误决策的条件** 根据 `SubsetSumSolver` 的决策逻辑，算法之所以会"跳过"`s*`，是因为未能满足其 `if` 判断条件。该条件为 `sᵢ <= T_rem` 且 `T_rem - sᵢ <= MaxSubsetSum(S_rem)`。

   分析这个条件对于 `s*` 不成立的原因：
   - **对于 `s* <= T_rem`**：在处理 `s*` 时，`T_rem` 的值等于 `T` 减去所有已被选中的、比 `s*` 更大的 `S*` 成员之和(由S降序保证)。

   因此，`T_rem` 的值恰好等于 `s*` 加上所有比 `s*` 小的 `S*` 成员之和。

   即

   $$T_{rem} - s^* = \sum_{x \in S^, x < s^} x$$

   所以，`T_rem >= s*` 必然成立。
   - **因此，决策失败的唯一原因**，只能是算法在那个时刻判断出：

$$T_{rem} - s^* > \text{MaxSubsetSum}(S_{rem})$$

4. **推导矛盾** 来分析这个不等式的两端：
   - **不等式左端 `T_rem - s*`**：根据上面的分析，这个值等于**最优解 `S*` 中，所有比 `s*` 小的元素的和。**
   - **不等式右端 `MaxSubsetSum(S_rem)`**：这个值是算法当前"待考察"的元素集合 `S_rem` 所能构成的最大子集和。
   - 最优解 `S*` 中那些比 `s*` 小的元素，必然是 `S_rem` 集合的一个子集。因此，从 `S_rem` 所能构成的最大子集和，必然大于或等于其任何一个子集的和。所以，逻辑上必然成立：

$$\text{MaxSubsetSum}(S_{rem}) \geq \sum_{x \in S^, *x* < *s*^} x

使用之前得到的等式, 也即是：

\text{MaxSubsetSum}(S_{rem}) \geq T_{rem} - s^*$$ 此时，矛盾出现：

- 我们的**假设**（算法失败）导出了结论：`T_{rem} - s* > MaxSubsetSum(S_{rem})`。
- 而**逻辑推演**得出的必然结论是：`T_{rem} - s* <= MaxSubsetSum(S_{rem})`。

同一个表达式不可能同时大于并小于等于另一个表达式。数学矛盾产生。

5. **得出结论** 产生矛盾的根源在于我们第一步的假设。因此，“存在一个解，但算法没有找到它”这个假设是错误的。

   故待证命题为真：对于任何存在解的子集和问题实例，`SubsetSumSolver` 算法保证能找到解，并以 `T_rem = 0` 的状态终止。

此证明确保了 `SubsetSumSolver` 算法的决策逻辑足够**完备**，不会遗漏任何存在的解。

所以，`SubsetSumSolver` 算法是**完全正确性**。

第三步：复杂度分析与矛盾推导

- **复杂度**：`SubsetSumSolver` 算法对集合 `S` 中的 `n` 个元素进行了一次遍历。在每次循环中，它调用了一次多项式时间的 `MaxSubsetSum` 函数。因此，整个 `SubsetSumSolver` 算法的运行时间是多项式的。
- **矛盾**：我们已经设计出一个多项式时间算法 (`SubsetSumSolver`)，它能准确地解决 NP-complete 的“子集和问题”。这直接导出了 P=NP 的结论，与计算复杂性理论中 P≠NP 的基本假设相矛盾。

第四步：结论

导致上述矛盾的根源在于我们在第一步中的假设——即“`CARD_GAME_OPT` 存在多项式时间解法”。因此，该假设必须是错误的。

所以，卡牌游戏优化问题 `CARD_GAME_OPT` 是 NP-hard 的。

结论2：卡牌游戏得分的泛化版本最大化问题是 NP-hard 的。

又根据之前的结论,strategy_TrueScore函数的步骤是 排序数组B->遍历B的每一个元素-> 检查数组A和当前遍历的B元素是否匹配(有交集)->计算匹配生成的连续子序列的和

```
数组 A 的大小为 m

数组 B 的大小为 n
```

排序时间复杂度为 O(n log n)

线性扫描A与B[i]是否满足匹配规则 时间复杂度为 O(n)

计算匹配生成的连续子序列的和(打出这张牌的得分) 时间复杂度为 O(n)

累加B的所有元素 对应的得分,得到得分总和 时间复杂度为 O(n)

总的时间复杂度为 O(n log n) + m *(O(n)+O(n)+O(n)) = O(n log n+O(mn))

它是多项式时间复杂度的,所以得到

结论3:卡牌游戏得分的泛化版本最大化问题是NP的

根据结论2和结论3,

卡牌游戏得分的泛化版本最大化问题是NP-Complete的

算法优化讨论/代理模型的提出

问题的NP-Hard性质导致了可能的决策空间非常大,并且我们已经证明了在当前length(A)=6,length (B)=3的小规模下的决策空间依然巨大,所以在编程时很难使用缓存技术来加速,因为缓存难以命中

动态规划:

A 的状态在每次插入后都会改变，并且列表内容本身复杂，而且导致了难以找到重叠子问题,难以作为 DP 状态的键进行存储和查找, A 的动态状态是问题所在。因此，动态规划 在这个针对这个问题并不适用。

剪枝/启发式方法:

基于对于simulate_insertion_tool的匹配规则的理解,在特定的规模,的确可以使用启发式方法优化,来跳过一些递归分支, 我定义了future_score 这个特征来实现 具体内容在遗传算法的实现中展开

代理模型（surrogate modeling）：在这个过程中，这个Recursive_StrategyAndScore算法生成结果数据，然后将这些数据用作训练集来训练AI模型。训练完成后，AI模型学习到了原始算法的行为模式，可以在允许一定的误差的情况下预测该算法在新输入上的输出结果，而无需实际运行原始算法。 这就是本文着重要讨论的内容

第二部分 AI 技术细节

软件与硬件介绍

```
硬件 (Hardware):
    CPU: AMD Ryzen 9 5900HS (8 Cores, 16 Threads)
    GPU: NVIDIA GeForce RTX 3050 Laptop (4GB VRAM)
    内存 (RAM): 16 GB
软件 (Software):
    操作系统 (OS): Windows 11
    IDE: PyCharm
    版本控制 (Version Control): Git
    环境管理 (Environment Management): conda 25.1.1
    Python版本:  3.12.9
```

> (注:针对这个问题而言,4G的显存并不是它的瓶颈 虽然它的确影响了我的参数配置的抉择,但是对于本文的模型,硬件是足够的)

模型选择

我选择了transformer[3]和遗传算法[4]作为ai模型

使用遗传算法的原因 其一是这个决策序列空间是巨大的，并且每一步的决策都依赖于上一步的结果。遗传算法是擅长在复杂、高维、非连续或目标函数难以微分的搜索空间中寻找最优解。 其二是它相对容易实现,而且它的计算成本低,可以快速得到结果。

选择使用transformer也同样是决策会相互影响的原因,这个问题的3次出牌策略中,每一次出牌都会产生动态的A,所以导致影响了之后的出牌策略,Transformer 使用自注意力机制（self-attention），能够捕捉序列中任意两个元素之间的关系(比如第一次插入是怎么影响第二次和第三次策略的)。这对于取得最佳得分的出牌策略非常重要

此外还有一个原因,Transformer的计算是GPU密集的 而GA的计算是CPU密集的,将它们集成可以实现硬件层面的并行,在离线计算时可以充分利用硬件性能

我使用递归穷举算法生成训练集来训练遗传算法和transformer模型,最后集成得出最后的代理模型

1.训练集的获取以及预处理

我测试了多种大小的数据集,最终决定用大约10k条数据集训练遗传算法,并使用大约5M条数据训练transformer模型 因为对于遗传算法而言,数据集并没有全部参与训练 ,遗传算法只会从数据集抽样进行遗传和评估,所以这里的数据集只需要够均匀即可,对于transformer而言,数据集的大小的确能提升模型性能,但是考虑到训练的时长和硬件限制,经过测试,我觉得5M条数据集应该足够了,再大的数据集难以操作而且对模型的提升并不明显

需要强调的是,这里有一个显著的区别 遗传算法和transformer模型使用训练集的方法是不同的

让我们再回顾一下训练集样本

样本示例: { "A": [10, 3, 4, 2, 12, 9], "B": [3, 8, 4], "max_score": 55, "best_moves": [[0, 0], [1, 1], [2, 5]] }

遗传算法的思路是遍历每一个可插入位置,并使用基因组去计算该插入位置的价值 训练集的价值在提供了固定的一批A,B,便于基因的评估和筛选 它本质是启发式方法,还是在遍历穷举 ,然后用自己的一套评分逻辑去评估该策略(将B[i]插入A中的某个位置)的价值,接着使用贪心算法找出价值最高的 它并没有使用到训练集的max_score,和best_moves

transformer模型则是使用了训练集的A,B和best_moves 它是"观察"大量的A,B,best_moves后,然后学着找出(A,B)|-> best_moves的映射关系 它并没有在穷举,或许它恰好找到的映射关系就等价于之前提到的穷举递归?或者是一个精妙的算法?但这很难判断,构建出神经网络的很可能极其复杂,这一部分我们在 Transformer的建模范式冲突与归纳偏置分析讨论 中进一步展开分析

数据集生成代码

generate_training_data(num_samples=10000) (实现见code_path)

这里有一个要点:

bestmoves的长度未必都是3 因为当B中就算再出牌也不能得分时(比如将B中的3张牌出了两张后,剩下的一张与A无法产生匹配) recursive_StrategyAndScore不会将移动加入到策略中,检查输出的best_moves的长度后,发现这样的情况大概占比num_samples的10%

所以现在有一个需要抉择的地方：对于这些长度不是严格为3的best_move 是应该跳过它们,还是填充它们到3?跳过的话会丢失重要的内在关系吗?如果填充会反而引入噪声吗? 这需要结合训练出的模型的预测结果来判断究竟哪个好 经过后续实验验证，我们发现采用填充策略的模型表现更优,具体对比分析详见第二部分的 ***数据集预处理 "填充"和"跳过"*** 讨论

所以我们现默认,数据集经过了填充处理 , 具体分析的细节放在ai模型的评测中再详细讨论

由于A,B是使用deal_cards_tool随机生成的,需要对数据集去重,可以使用python的set完成,接着需要详细检查数据的格式,异常处理 等 (代码详见dedpu.py)

最后得出给遗传算法使用的数据集data_GA.json 样本数为10k 这是一个样本示例: { "A": [10, 3, 4, 2, 12, 9], "B": [3, 8, 4], "max_score": 55, "best_moves": [[0, 0], [1, 1], [2, 5]] }

给transformer使用的数据集data_TR.json的处理也是一样的,只是由于数据过大,使用了多进程加速获取数据集,以及后续将json格式转换jsonl以支持流式加载 样本数在去重前预设为5M

2.1 遗传算法的实现

遗传算法框架概述

基因的组成

- `calculate_future_score` :用启发式方法计算未来得分这个特征 6维权重向量，对应6个游戏特征：当前得分、移除长度、新长度、未来得分、新A的元素总和、交集数量。

种群初始化

- 种群规模500个个体
- 70%按启发式范围初始化，30%完全随机
- 引入多样性防止早期收敛

基因组的选择与繁殖

- 精英保留：10%最优个体直接传承
- 锦标赛选择：竞争性选择父代
- 交叉：父代权重加权平均
- 变异：添加高斯噪声，变异率自适应调整

进化策略

使用了 传统的遗传算法 和岛屿模型算法[5] 这两种进化策略

- `island_model_evolution()`: 岛屿模型，将种群分割为独立岛屿，定期迁移优秀个体
- 动态方法切换： 根据历史表现自适应选择进化方法

适应度评估

- `evaluate_genome()`: 用个体权重执行多轮游戏，返回平均得分
- `GA_Strategy()`: 基于权重计算最优插牌策略，使用特征向量与权重的点积评估每个可能位置

优化机制

- GPU加速：PyTorch张量运算
- 并行评估：多进程处理不同个体
- 批量处理：减少GPU调用开销
- 早停机制：连续无改善时提前终止

训练配置

- 种群大小：500
- 进化代数：20
- 评估轮数：200轮/个体

输出结果

最优个体的6个权重参数，构成AI的决策策略模型。

特征设计

根据卡牌的记分方法来决定需要使用的特征

```
features = [
    score,  # 当前策略的子集得分
    removal_length,  # 匹配的移除长度
    new_length,  # A由于插入操作得到的新长度
    future_score,  # 未来得分
    sum_new_A,  # 匹配后新A的元素总和
    B_card_proportion,  # B的出牌占B剩余牌的价值比例
]
```

遗传算法的genome 是一个包含6个浮点数的列表。这6个数字代表了评估游戏决策时所用到的6个特征的权重

当然这些特征的值是显然比较大的,需要进行特征缩放

而且由于这些特征都是动态生成的,无法事先使用标准化处理

这里在将所有特征转换为张量后 在线的计算了均值和方差 然后标准化

具体实现见 code_path 中GA_Strategy.py里的做法

"future_score"特征

特征里有新意的是future_score 由于B中的牌数量较少(0~3) 所以可以根据这个特点进行启发式算法, 这里的future_score意思是 由于每次要从B中选一张插入A,而且每次的选取,插入都可能造成A的变化,然后导致后续的选

取,插入步骤的最优得分策略变化 future_score指每次选取插入这个操作之后,剩余牌可能的得分 这是一个很重要的特征 它动态的评估了未来的卡牌状态 就好像象棋职业玩家会考虑某一步对以后的棋子策略带来的影响一样

其他的特征选取于simulate_insertion_tool的返回值 match_found和removal_length有重合,考虑到模型复杂度,它没有被作为特征

这个计算future_score的启发式函数的思路是这样的,从B中的牌抽取一张插入A时,同时考虑在这次选取插入操作实现之后:

1 当如果 A 和 B 没有任何重复元素,那么future_score是0

2 如果没有剩余的B玩家的牌,那么future_score是0

3 如果B中只剩下一张牌,检查这张牌和A是否有交集 如果有 则调用get_best_insertion_score计算得分作为future_score,如果没有那么future_score是0

以此定义未来得分函数 calculate_future_score(A, remaining_B) (实现见code_path)

遗传算法核心函数:

```
island_model_evolution(population, fitnesses, pop_size, tournament_size,
mutation_strength,
                           num_rounds, islands=3, migration_rate=0.1,
                           generation=0, max_generations=60)(实现见code_path)
"""
返回由于岛屿模型进化得到的新的种群
"""
```

```
GA_Strategy(genome, A, B)(实现见code_path)
"""
根据A,B和给定的基因生成策略 (给调用遗传算法提供接口)
计算每一个插入位置的特征张量,并与它和基因张量做点积计算 得到这个位置的"评估值",并按照评估值从大到小排序

返回根据排序后的顺序生成策略
"""
```

```
evaluate_genome(genome, num_rounds=1000, seed_base=111)(实现见code_path)
"""
基因评估函数
用一批种子生成的A,B来评估基因是否够好
返回计算fitness的平均值
"""
```

```
###遗传算法训练主函数
genetic_algorithm(pop_size, generations, num_rounds, elitism_ratio,
tournament_size,
                      evolution_methods,
                      early_stop_generations, early_stop_threshold)(实现见
code_path)
```

根据以上遗传算法有关过程训练,返回得到的最佳基因组

```
#### 训练参数

遗传算法的实际参数  经过测试,这些参数已经能足够好的结果了:

    pop_size=500
    generations=25
    num_rounds=500
    elitism_ratio=0.1
    tournament_size=3
    evolution_methods=['island','standard']

    early_stop_generations=3
    early_stop_threshold=0.01

    genome = genetic_algorithm(pop_size,generations, num_rounds, elitism_ratio,
tournament_size, evolution_methods, early_stop_generations, early_stop_threshold)
```

基因分析

接着我们来分析一下得到的最佳基因 以下基于我的一个测试数据

这是一个训练后得到的基因 ,模型在第16代由于早停终止

```
[2.5638819522632654, 0.30592210650063667, -1.1848400755378474, 4.178412919525965,
-1.4191214580170772, -0.09006199513519475]



各进化方法性能分析:

island方法: 平均适应度=49.33, 最大适应度=49.90, 改进率=0.14%, 使用次数=6
standard方法: 平均适应度=49.50, 最大适应度=50.30, 改进率=4.27%, 使用次数=9
```

这说明标准遗传算法和岛屿模型进化都起到作用的,

我进行过多次测试,两种方法不一定哪个改进率高,说明对于不同基因组,不同的测试集,适用的模型不同,并且岛屿模型在这个问题里并没有明显的优越性

最终改进率都较低,说明第一代基因表现的已经不错了(这和温启动也有关)

```
基因组特征权重:
当前得分: 2.5639
移除长度: 0.3059
新长度: -1.1848
未来得分: 4.1784
新A元素总和: -1.4191
出牌占B剩余牌的价值比例: -0.0901
```

回到基因本身,分析它的最终特征的权重:

特征1: 当前得分 (2.5639)

这个权重非常高且为正值，表明算法非常重视当前操作能够直接带来的得分

特征2: 该位置的匹配后移除的长度 (0.3059)

这个权重为低正值，表明算法倾向于移除较少的元素。表明算法倾向于避免移除过多的元素。这可能是因为保留更多元素可以为未来的移动提供更多可能性。

特征3: 新长度 (-1.1848)

负值，表明算法倾向于在插入匹配操作后得到较短的新A。这似乎与特征2的权重倾向吻合

(虽然特征2和特征3的值可以通过简单的数学计算得到(新A的长度=旧A的长度-由于匹配被移除的长度) 但是我无法保证ai算法能察觉到这个关系 而且这两个状态是代表了不同的视角 所以我倾向保留 这个特征可能带来性能提升或加速模型收敛)

特征4: 未来得分 (4.1784)

这个权重非常高且为正值，这表明算法非常重视移动对未来可能得分的影响，具有很强的前瞻性。

特征5: 新A的元素总和 (-1.4191)

权重为较高的负值 可能说明 算法不仅仅考虑当前得分 并且保留比较少的得分可能性可能让后续的操作获得

特征6:被移动的牌占B中剩余牌的价值比例(-0.0901)

这个特征表明了算法是倾向于先打出高价值的牌,还是先打出低价值的牌 这次的数据是接近0的权重,影响非常小. 但是某些测试中 这个值小于-1 说明此特征或许与其他特征能做到一些平衡

然后要调用接口(GA_strategy)来测试 在给定真实的A,B的情况下,评估我们的遗传算法和得到的基因 但是这一部分会放到后面,和transformer一起评测

2.2 transformer模型算法的实现

我使用了标准的transformer模型,训练时会输入A,B,以及best_moves 在预测时,A,B是输入特征, best_move是目标,用于计算损失函数，指导模型优化(监督学习的目标标签)

这个transformer的训练,我一开始尝试过较精简的实现 但是出现了收敛过慢,梯度爆炸,无法灵活的手动停止训练等等问题 所以除了早停和梯度裁剪之外,我加入了学习率预热,余弦退火,优雅退出*/信号处理,来在不破坏性能的前提下加速训练

> *:(优雅退出在最开始探索训练参数大概范围时有很大的帮助,比如loss足够小了我就可以手动停止训练,继续训练不但费时,而且有梯度爆炸的风险 此时获取够好的临时模型再分析,可以用于推断更好的参数范围 虽然在最佳训练参数基本确定后就不需要手动停止,优雅退出也没什么作用了)

transformer模型框架概述

数据准备与特征工程

```
    prepare_data_transformer(sample, num_a=6, num_b=3) 函数:
        关键的数据预处理和特征构建步骤。
        将原始的 A、B 序列和对应的 best_moves 转换为模型训练所需的格式。

        特征工程 (input_dim=6): 为A和B序列中的每个元素创建6个特征。



        输出：input_sequence (形状 (9, 6)) 和对应的 order_target、pos_target 标签。
```

训练主流程

```
        总控模型训练过程。
        初始化: 模型 (TransformerMovePredictor)、AdamW 优化器。
        损失函数:
            order_criterion (nn.CrossEntropyLoss): 用于B序列顺序预测。
            pos_criterion (nn.MSELoss): 用于B序列元素位置预测。
        学习率策略:
            lr_warmup: 训练初期学习率线性预热。
            CosineAnnealingLR: 预热后使用余弦退火策略调整学习率。
        训练循环:
            数据打乱与批处理。
            调用 prepare_data_transformer 处理每个批次的数据。
            前向传播、计算总损失 (顺序损失 + 位置损失)。
            反向传播、梯度裁剪、优化器更新参数。
        早停机制: 若连续多个 epoch 损失无明显改善，则提前终止训练，并保存最佳模型。
        模型保存: 训练结束或中断时，保存模型状态字典到 .pth 文件。
    signal_handler(...): 允许通过 Ctrl+C 安全中断训练并保存模型。
```

模型预测与后处理

```
Transformer_predict(A, B, model) 函数:

 预测并返回最佳策略列表, 预测得分
```

特征工程策略

A序列特征 (目标序列)

```
对于A的某个元素a(数字)而言:

[a占A的元素总和的比例, 是否是极值, a标准化后的值, a标准化后的值, a归一化后的值, A与B交集大小占A元素数的比例]
```

标准化后的值: 体现a数值与A的均值之间的差距

归一化后的值: 0-1之间 直接体现a的数值

B序列特征 (插入序列)

对于B的某个元素b而言:

```
[b占B的元素总和的比例,  未来得分与(A中元素的和+B中元素的和)的比,  b在A是否有交集, b在A中的相对位置（b与A没有交集则为-0.1）,  B中重复元素占所有B元素数的比例,是否是极值]
```

输出结果

```
训练完成后，生成一个 .pth 文件,包含训练好的 TransformerMovePredictor类型的权重参数。可用于后续的策略预测。
```

这和GA的动态特征计算不同,这里的特征只对初始的A,B计算一次 这是一个一次性（One-Shot）预测

transformer核心函数(大多数是transformer的标准实现):

PositionalEncoding类 (实现见code_path) 将输入的词嵌入向量与从缓冲区中取出的、长度匹配的位置编码向量相加，再返回dropout层的处理结果,而为模型注入关键的序列顺序信息

TransformerMovePredictor类 (实现见code_path) 训练好的模型就是一个TransformerMovePredictor类的实例,训练会改变它的所有神经网络层的权重和偏置,然后使用两个预测头,返回预测B中元素的操作顺序 和 代表每个 B 元素预测要插入的A的哪个位置

prepare_data_transformer(sample: dict, num_a=6, num_b=3) (实现见code_path) 负责数据预处理和特征工程 将原始的 A、B 序列和对应的 best_moves 添加特征后,一起打包转换为模型训练所需的格式并返回。

Transformer_predict(A, B, model, num_a=6, num_b=3) (实现见code_path)

使用训练好的 Transformer 模型进行预测 (针对 A=6, B=3)。

返回值: (最佳策略列表, 预测得分)
这里使用了一个启发式的判断 如果A和B没有交集，且B中没有重复元素 说明无法得分 这时候不需要进行预测,直接返回默认移动策略(事实上训练模型时，已经跳过了A,B没有交集的样本)

A和B没有交集无法得分很好理解 对于AB没有交集,且B中如果有重复元素 b1=b2,那么b1与b2可以分别插到A中,匹配规则依然成立,甚至如果b1插入A的第一个位置,b2插入最后一个位置 能获得sum(A)+sum(B)的得分

训练主函数:

train_model(train_data, epochs, batch_size, model_path="transformer.pth", num_a=6, num_b=3, warmup_epochs, lr_max, lr_min, patience, min_delta) (实现见code_path)

以下是实际调用的参数(patience=2看似激进,但是它经过测试是合适的值)

```
train_model(train_data, epochs=100, batch_size=2048,
model_path="transformer_move_predictor.pth",
            num_a=6, num_b=3, warmup_epochs=5, lr_max=0.0005, lr_min=0.0000001,
            patience=2, min_delta=0.01)
```

在Epoch =12, LR: 0.0000001 时,由于早停终止训练 Average Loss约为 0.04

transformer生成的pth格式模型有很强的黑箱性质 所以这里我们不直接分析它

现在我们完成了遗传算法和transformer 的训练 接着我们要来比较这两个模型得到的结果

评估流程介绍

我是这样评估两个模型的:

整个评估流程由核心函数 Compare_TwoModel(model, other_model, rounds) (实现见code_path) 驱动，可以分解为以下几个关键步骤：

1. 初始化与循环测试

   设定测试轮数 (rounds): 函数接收一个 rounds 参数，决定了要进行多少轮独立的测试 这里的测试次数是20000 循环生成测试用例: 在每一轮循环中，通过 deal_cards_tool() 函数 (实现见code_path) 生成一组新的测试数据 A 和 B。每一轮的数据都是随机独立的，这保证了模型评估的泛化能力。

2. 模型预测与性能记录

对于每一轮生成的 A 和 B：

调用模型获取策略:
    TestModel_with_given_A_B(model, A, B) 函数 (实现见code_path) 被分别用来测试 genome 模型和 Transformer 模型。

在这个函数内部，每个模型会根据当前的 A 和 B 状态，预测出一个三步操作的最佳策略 move。这个 move 定义了应该先出 B 中的哪张牌，以及插入到 A 中的哪个位置。
计算真实得分:
模型返回的仅仅是它认为最好的策略（move）。而不是使用模型自己预测的分数。

使用 strategy_TrueScore(A, B, move) 函数 (实现见code_path) ，这是依据move计算分数的函数,根据模型返回的 move，一步一步地模拟整个出牌过程，并计算出这个策略能够得到的 真实总分。
记录运行时间: TestModel_with_given_A_B 函数 会精确记录每个模型从接收数据到返回策略所花费的时间，用于评估模型的计算效率。

3. 获取"真实值"（Ground Truth）

在每一轮中，代码还会调用之前提到过的 recursive_StrategyAndScore() 函数通过 穷举递归搜索 的方式，计算出当前 A 和 B 组合下能够得到的 理论最高分（true_score） 和达成该分数的对应的 最优策略（true_move）。

以可能达到的最佳情况来评估两个模型

4. 数据汇总与分析

当所有 rounds 完成后，代码收集了三组核心数据：genome模型的得分列表、Transformer模型的得分列表，以及每一轮的理论最高分列表。接下来进行详细的数据分析：

描述性统计 (compute_statistics):
对每个模型得分列表和真实值列表，计算并打印出基础统计数据：
平均值 (mean): 模型的平均性能。
中位数 (median): 排除极端值影响后的典型性能。
标准差 (std_dev): 模型性能的稳定性。标准差越小，说明模型性能越稳定。
最大/最小值 (min/max): 模型的性能上下限。

均方误差 (MSE):
计算每个模型的得分与真实最高分之间的均方误差。MSE 能很好地度量模型预测的偏差大小，值越小说明模型结果越接近最优解。

推断性统计检验 (compute_statistical_tests):

分析模型得分与真实值之间的 偏差 (Bias)。
配对 t-检验 (Paired t-test):
用于检验"模型的平均得分"与"真实值的平均分"之间的差异是否 统计上显著。
p-value 是核心指标。如果 p < 0.05，则认为模型得分与真实值之间的差异是显著的，而不是由随机因素造成的。
效应量分析 (Cohen's d):
t-检验只告诉我们差异是否"显著"，但没有说明这个差异有多"大"。
Cohen's d 用来衡量差异的 实际大小或影响程度（极小、较小、中等、较大）。

可视化分析

1. 小提琴图 (Violin Plot)

目的: 展示三组数据（真实值、算法1得分、算法2得分）的 完整分布情况。

2. 偏差条形图 (Bias Bar Chart)

   目的: 观察模型在不同得分分数段的偏差规律 和每个分数段的样本数

   设计偏差偏差条形图,将纵轴的刻度与小提琴图对齐,借助小提琴图能直观的展示了预测分数的分布情况的特性, 很容易直观的看到落在某一个分数段的预测数据在所有预测数据的分布情况,并且了解这一分数段的平均偏差

数据集预处理 "填充"和"跳过"

现在我们可以回到对于训练集中长度不是严格为3的best_move的处理问题 (这是由穷举递归的 比如操作了第二步后,第三步再怎么出牌都无法得分,这时候返回的best_move为的长度为2 而不是3 训练集中长度不是严格为3的best_move的情况经过测试大概占10%)

获取训练集 代码

def generate_training_data(num_samples=10000): dataset = []

```
for i in range(num_samples):


    #初始化A, B
    A, B = deal_cards_tool()

    max_score, best_moves = recursive_StrategyAndScore(A, B)
```

#保证best_move这个二维数组的3个子数组的中第一个元素是互异的 (因为只能操作3次,每次只能是0 1 2其中之一,且每次操作的序号是唯一的) whole = [0, 1, 2] current_rows = set(move[0] for move in best_moves)

```
    remaining = [i for i in whole if i not in current_rows]
```

""" 这里使用的是"填充"实现

如果使用"跳过"的策略 可以改成 if len(best_moves) != 3: continue """ for i in remaining: if len(best_moves) < 3: best_moves.append([i, 0])

```
    dataset.append({"A": A, "B": B, "max_score": max_score, "best_moves":
best_moves})

with open("data.json", "w") as f:
    json.dump(dataset, f, indent=4)
```

generate_training_data(num_samples=10000)

接着分别使用填充/跳过的方法训练模型,并比较

以下数据基于我的实验结果

基于数据集预处理使用"填充"的讨论

我们使用这个函数为遗传算法和transformer分别生成训练集

data_GA_fill.json 和 data_transformer_fill.json

去重处理

data_GA_fill.json找不到重复的样本

data_transformer_fill.json --- 去重结果 --- 原始记录数: 5000000 去重后记录数: 4989455 移除的重复记录数: 10545 重复率约为0.21% 这是极小的 也验证了之前算法部分讨论的"决策空间巨大"和难以设计缓存的问题

由于5M的数据比较大 所以这里我转换成了jsonl格式，有助于提升效率

这里算法1代表遗传算法模型 算法2代表Transformer模型

运行比较函数,测试轮数为5000

得出以下统计数据

统计数据对比分析表

测试配置： 20000轮测试
比较对象： 算法1(GA) vs 算法2(transformer) vs 真实值

算法性能对比

表1.

指标	算法1 (GA)	算法2 (Transformer)	真实值
平均得分	52.52	58.71	59.41
中位数得分	53.00	59.00	59.00
得分标准差	12.88	11.76	11.27
最低得分	4	2	23
最高得分	95	96	96
运行时间 (秒)	38.38	80.71	-
均方误差 (MSE)	102.3389	14.23675	-

偏差分析 (真实值 - 预测值)	算法1 (GA)	算法2 (Transformer)
均值偏差	-6.89	-0.70

偏差分析 (真实值 - 预测值)	算法1 (GA)	算法2 (Transformer)
中位数偏差	-5.00	0.00
标准差偏差	7.40	3.71
最低得分偏差	-42	-53
最高得分偏差	0	0

统计检验与效应量分析	算法1 (GA)	算法2 (Transformer)
t 统计量	-131.689	-26.784
p 值	0.000e+00	2.679e-155
统计显著性	显著 (p < 0.05)	显著 (p < 0.05)
Cohen's d	-0.931	-0.189
影响程度	较大	极小

*所有偏差均按照真实值相应数据减去预测值相应数据的方式计算

综合性能分析

从实验数据来看，Transformer (算法2) 在多个关键指标上显著优于 GA (算法1) ，尤其是在预测精度和偏差控制方面表现更为出色

1. 预测精度与误差

   均方误差 (MSE) Transformer 的 MSE 为 14.23 ，远低于 GA 的 102.33 ，表明 Transformer 的预测结果更接近真实值，具有更高的精度。 偏差分析进一步支持了这一点：Transformer 的均值偏差仅为 -0.70 ，而 GA 的均值偏差高达 -6.89 。此外，Transformer 的中位数偏差为 0.00 ，说明其预测值的分布中心与真实值完全一致，而 GA 的中位数偏差为 -5.00 ，存在明显的系统性偏差。

2. 运行效率

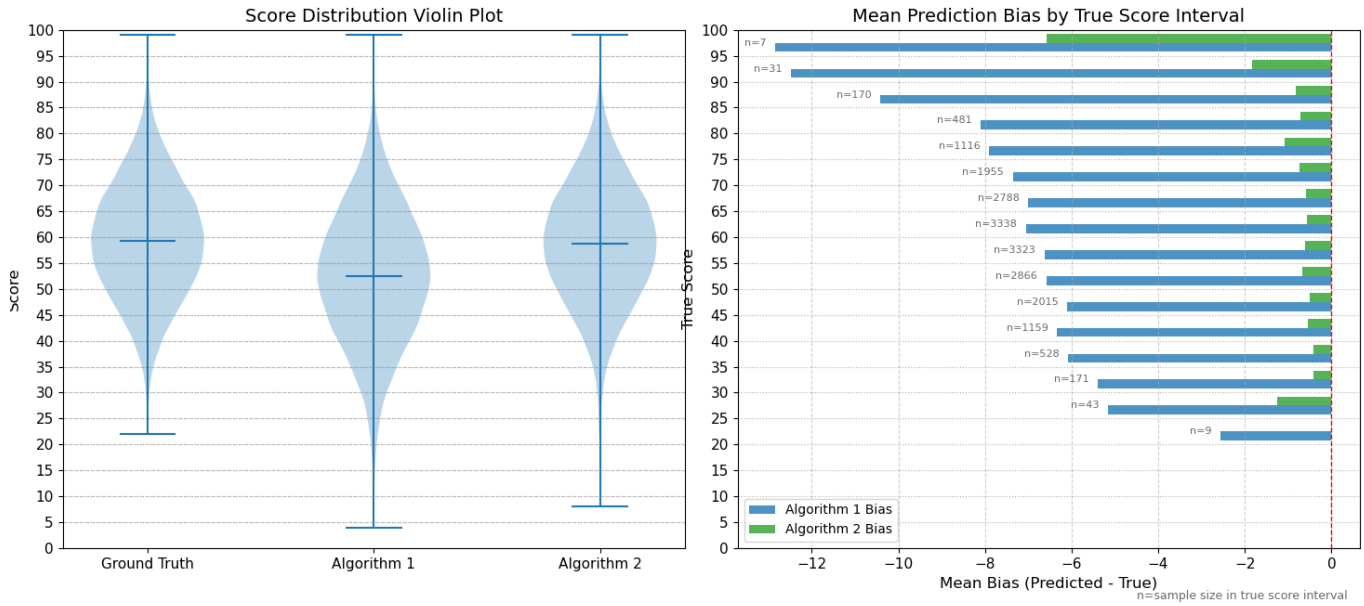
   在运行时间方面，GA 的平均运行时间为 38.38 秒 ，而 Transformer 的运行时间为 80.71 秒 运行时间比是 1:2.07 GA的运行速度远高于 Transformer

3. 统计显著性与效应量

   配对 t 检验结果显示，两种算法的预测值与真实值之间的差异均具有统计显著性（ p < 0.05 ）。然而，效应量分析（ Cohen's d ）揭示了两者的实际影响程度：GA 的 Cohen's d 为 -0.931 ，表示其预测值与真实值之间存在较大偏差；而 Transformer 的 Cohen's d 仅为 -0.189 ，表明其预测值与真实值之间的偏差极小，几乎可以忽略不计。

4. 极端值处理

   在最低得分的预测上，两种算法均存在一定偏差，尤其是 Transformer 的最低得分偏差达到了 -53 ，比 GA 的 -42 更大。这可能表明 Transformer 在极端值的预测上仍需改进。不过，最高得分的预测偏差均为 0 ，说明两种算法在高分段的预测能力相当。



左图：分数分布小提琴图 (Score Distribution Violin Plot)

这个图表展示了真实分数以及两种算法预测分数的整体分布情况。

> 形状宽度：表示数据在该分数上的密集程度。越宽的地方，数据点越多。
>
> 内部箱形：中间的横线是中位数，箱体代表了25%到75%的数据范围（四分位距）。

分析要点：

> 真实数据 (Ground Truth)：分数的分布相对集中在40到80分之间，中位数大约在59分。这是一个基准，理想的算法应该生成与之相似的分布。
>
> 算法1 (GA)：
>
> 其分布形状与真实数据相似，但整体低分方向偏移。
>
> 它的中位数明显低于真实数据，大约在53分左右。
>
> 这表明算法1(GA)系统性地低估了分数，其预测结果普遍低于真实值。
>
> 算法2 (Transformer)：
>
> 其分布形状和位置与真实数据高度吻合。
>
> 它的中位数与真实数据的中位数几乎重合，都在59分附近。
>
> 这表明算法2(Transformer)的预测分数在整体分布上与真实情况非常接近，没有明显的系统性偏差。

小结： 从分数分布来看，算法2(Transformer)很好的地复现了真实数据的分布特征，而算法1(GA)存在明显的低估倾向。 但二者的预测都存在非常少量的最低分极端值

右图：按真实分数区间的平均预测偏差 (Mean Prediction Bias by True Score Interval)

这个图表更深入地分析了偏差，它将数据按真实的得分（Y轴）划分成多个5分的区间，然后计算在每个区间内，算法预测值与真实值的平均差异（即平均偏差 = 预测值 - 真实值）。

X轴：表示平均偏差。

负值：表示低估（预测值 < 真实值）。

0值（右侧的红色虚线）：代表无偏差，是理想状态。

*注 :与真实值的偏差都是负的是因为这是按照模型预测的操作,使用插入模拟函数 simulate_insertion_tool(代码见code_path) 获得的得分,这个值必然＜=理论最佳得分,所以算法只可能低估得分,x轴的值只可能是非正数

Y轴：真实的得分区间。

条形：蓝色代表算法1(GA)的偏差，绿色代表算法2(Transformer)的偏差。

n：表示每个分数区间内的样本数量，样本量越大，该区间的偏差结果越可靠。

分析要点：

算法1(GA) (蓝色条形)：

GA的预测偏差情况随着分数的提升大体上成正比关系

算法2(Transformer) (绿色条形)：

绿色条都接近0值线，表明其偏差很小。

在大部分分数区间，算法2只有轻微的低估,但在低分段（如25-30分）和极高分段（如95-100分）中，其偏差相对严重

与算法1(GA)相比，算法2(Transformer)的偏差控制得较好

小结： 偏差分析图证实了小提琴图的结论。GA相对于Transformer存在普遍的低估问题，而算法2的预测偏差很小。

总览与核心结论

算法2在预测准确性上显著优于算法1，是更优的选择。 尽管算法2的运行时间远长于算法1，但其预测结果在各项核心指标上都与真实值较吻合，误差较小。

它们有共同的缺点,对于极值的预测并不非常好 我觉得这是训练集数据分布边缘的极值点的比例导致的 ,并且不排除某些分数段偏高是遇到了分布外样本

现在我们要回到使用"跳过"方法来处理数据集 ,进行同样的数据统计

由于和"填充"的流程几乎完全相同,这里只做概述

比较"填充"与"跳过"对模型性能的影响(消融实验)

获取data_GA_skip.json 和 data_transformer_skip.jsonl数据集,并预处理(跳过move的长度≠3的样本后还剩下大概90%的数据,这里的5M数据集*90%已经够大,足以充分训练)

训练模型并保存 使用和"填充"方法相同的评测方法 得到 (算法1代表遗传算法 算法2代表transformer)

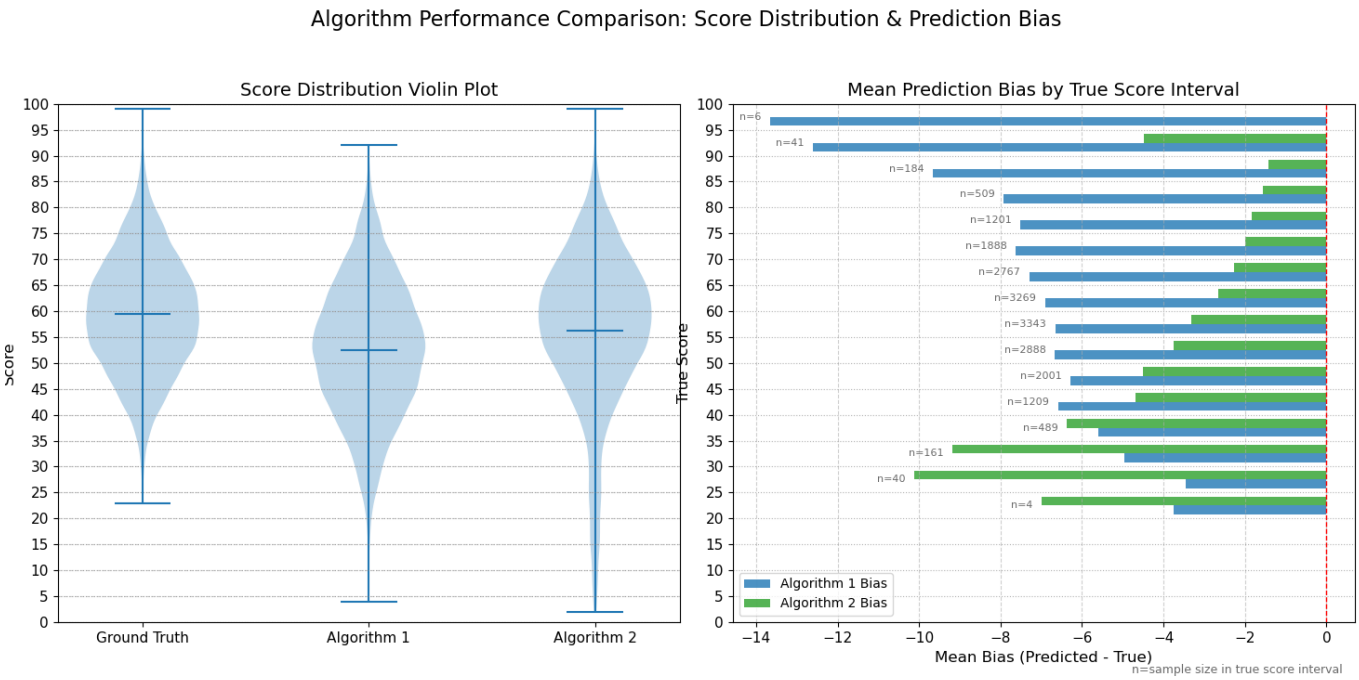
表2.

指标	算法1 (GA)	算法2 (Transformer)	真实值
平均得分	52.51	56.24	59.43
中位数得分	53.00	58.00	59.00
得分标准差	13.02	15.15	11.24
最低得分	4	2	23
最高得分	92	99	99
运行时间 (秒)	39.91	82.17	-
均方误差 (MSE)	103.87035	87.87605	-

偏差分析 (真实值 - 预测值)	算法1 (GA)	算法2 (Transformer)
均值偏差	-6.92	-3.19
中位数偏差	-5.00	0.00
标准差偏差	7.48	8.81
最低得分偏差	-38	-59
最高得分偏差	0	0

统计检验与效应量分析	算法1 (GA)	算法2 (Transformer)
t 统计量	-130.754	-51.232
p 值	0.000e+00	0.000e+00
统计显著性	显著 (p < 0.05)	显著 (p < 0.05)
Cohen's d	-0.925	-0.362
影响程度	较大	较小

对比使用"填充"策略的数据表1,得出实验结论: 使用"跳过"方式处理数据集后, 虽然算法1(遗传算法)各项数据受影响不大 但是算法2(transformer)的MSE明显提高了(14.24->87.88),平均得分和中位数得分降低,与真实值的偏差明变大,标准差降升高,效应量分析也从极小变成了较小



对比Figure_1.png,可以从图直观的观察到算法2(transformer)对于低分,比较之前的小提琴图与偏差条形图,中低分端产生了明显的更严重的偏差

我觉得这和遗传算法是基于启发的 所以怎么得出计算分数的方法是我们规定的 至于对训练集填充了0 可能让模型学会了识别这类"无论如何再怎么出牌都无法得分"情况,因此让transformer学习到了额外的特征,这些特性对低分段的预测有着重要作用("无论如何再怎么出牌都无法得分"的情况在低分段分布较多) 所以对结果产生了显著的影响

集成模型的提出

由于这两个模型是训练方法存在差异的,我猜想它们的关注点不同,当transformer预测出较差的结果,遗传算法或许比它好 而且遗传算法很快,集成遗传算法到Transformer并不会导致太多的额外时间消耗

所以为了验证这个猜想,我使用了这样一个方法 将Transformer和GA简单的集成在一起, 对于同一组A,B 这两个模型同时预测，计算这两种策略对应的真实得分,然后返回更二者中更大的得分

```
move1=Transformer_Strategy(A,B)

score1=strategy_TrueScore(A,B,move1)

move2=GA_Strategy(genome,A, B)

score2=strategy_TrueScore(A,B,move2)

if score1<score2:

    return move2
```

```
else:
    return move1
```

同样的我们来看现在的transformer+GA 集成模型与 Transformer模型(之前的算法2)的评测结果 重新测试 迭代次数:20000

指标	算法1 (GA+Transformer)	算法2 (Transformer)	真实值
平均得分	58.93	58.59	59.25
中位数得分	59.00	59.00	59.00
得分标准差	11.38	11.73	11.24
最低得分	16	5	21
最高得分	98	98	98
运行时间 (秒)	228.56	139.63	-
均方误差 (MSE)	3.6578	13.2122	-

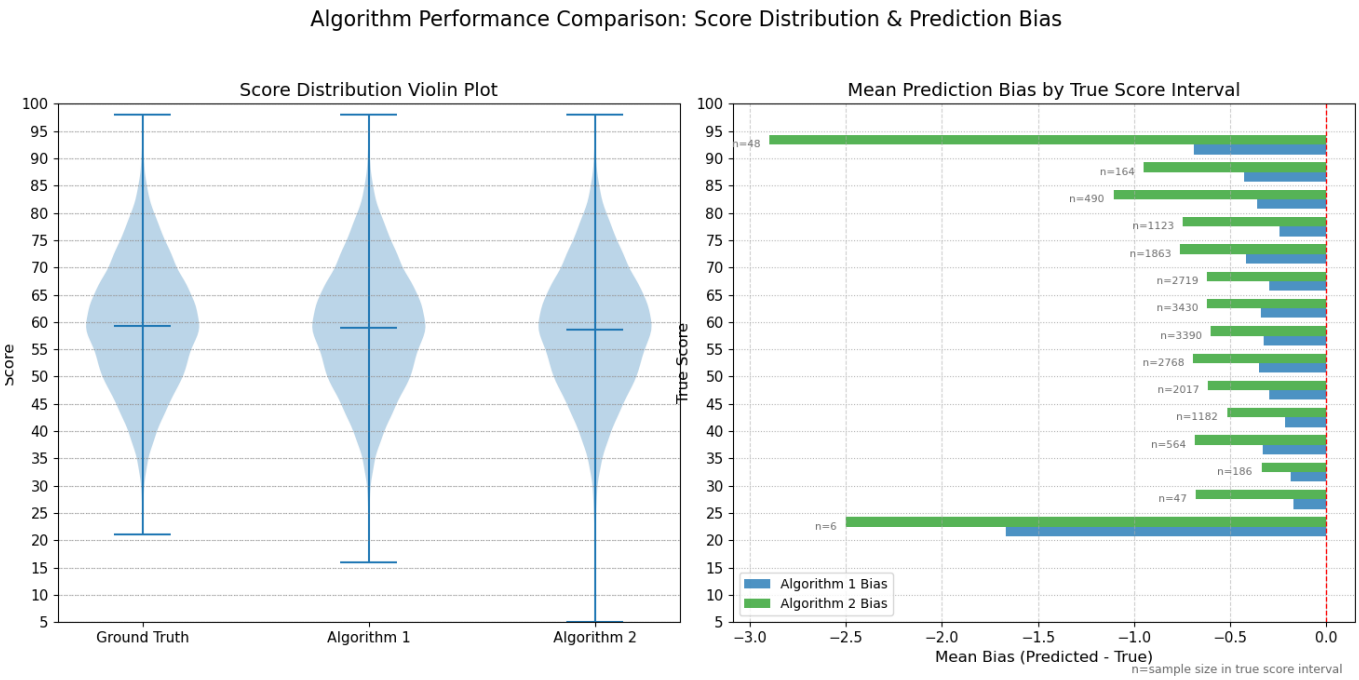
偏差分析 (真实值 - 预测值)	算法1 (GA+Transformer)	算法2 (Transformer)
均值偏差	-0.32	-0.66
中位数偏差	0.00	0.00
标准差偏差	1.88	3.57
最低得分偏差	-26	-52
最高得分偏差	0	0

统计检验与效应量分析	算法1 (GA+Transformer)	算法2 (Transformer)
t 统计量	-24.347	-26.153
p 值	4.661e-129	2.824e-148
统计显著性	显著 (p < 0.05)	显著 (p < 0.05)
Cohen's d	-0.172	-0.185
影响程度	极小	极小

分析:

- **偏差**: **算法1在各项偏差指标上均优于算法2**，其均值、标准差和最低得分的偏差都更小，说明其预测结果整体上更贴近真实数据。
- **统计检验**:
  - 两种算法的预测值与真实值之间的差异在统计上都是**显著的** (p < 0.05)，意味着这种差异不太可能由随机因素造成。
  - 然而，**Cohen's d** 分析显示，两种算法的效应量都**极小**。这说明虽然差异在统计上显著，但在实际应用中，这种差异的幅度非常微小。

比较结果,猜想是成立的 GA在20000次运行中辅助了transforme546次 虽然"辅助"的次数占比极低(546/20000≈0.027)，但是在各向指标几乎均有提升,精度(MSE)带来了显著影响(降低了72%), 由于串行和调用了strategy_TrueScore, Transformer集成 GA+Transformer后多了63% 的运行时间



通过观察小提琴图和偏差条形图能直观的感受到我们的集成模型是在预测准确度上的提升

集成模型的代码性能优化

现在我们来比较基于以上讨论 试着用得到的新得到的transformer+GA 模型在运行速度上去挑战原始的穷举递归算法(准确度我们之前已经分析了 并且穷举递归得到的就是真实值)

这里有很重要的一个注意点 这里的批处理讨论基于transformer+GA的离线计算且数据量大的情况 但是在需要低延迟,即时性,单次请求数据量小的即时处理场景(比如我们的游戏)是不适用的

对于这个Transformer+GA的集成模型 (Ensemble Model)

遗传算法 (GA)：其预测逻辑是贪婪且串行的，不适合进行批处理预测。

Transformer 模型：其模型架构和计算方式是并行的，非常适合进行使用GPU批处理预测(这也是神经网络的优点之一)。

实现思路

将Transformer的预测改为批处理

并使用ThreadPoolExecutor 启动两个线程，一个用于 GA，一个用于 Transforme 遗传算法的与其并行预测,GA是CPU密集的,transformer是GPU密集的,这样做很好的利用了硬件性能, 虽然代码会收到GIL(全局解释器锁)的影响 但是它依然能很好的提速

当使用 GPU 时：Transformer 推理任务主要由 GPU 执行，而 GA 任务由 CPU 执行。threading 协调了这两者，实现了 GPU 和 CPU 资源的真正并行使用，因为 GPU 计算不依赖于 GIL。

当仅使用 CPU 时：虽然两个任务都在 CPU 上，但由于 PyTorch 和 NumPy 库 ,在其核心计算部分会释放 GIL, GA 线程仍然可以在 Transformer 线程等待其 C++ 后端完成计算时获取GIL,获得执行机会。这虽然不是完美的硬件并行，但仍然比串行执行要快得多。

#批处理集成模型预测函数

def Transformer_predict_batch_plus_GA(A_batch, B_batch,genomeforassist, TR_model, num_a=6, num_b=3, device='cuda') (实现见code_path) """ 批量使用训练好的 Transformer 模型进行预测 (针对 A=6, B=3)。

```
参数:
    A_batch: 列表的列表，每个内部列表是一个A序列，长度为num_a
    B_batch: 列表的列表，每个内部列表是一个B序列，长度为num_b
    model: TransformerMovePredictor模型实例
    num_a: A序列的长度，默认为6
    num_b: B序列的长度，默认为3
    device: 计算设备，默认为'cuda'

返回:
    列表: moves_batch
"""
```

def execution_time() (实现见code_path)

使用20000个{A,B}样本测试递归算法,比较实现了批处理的Transformer+GA 未使用批处理的Transformer,串行的GA,穷举递归的运行时间

这里使用了一个小技巧,使用"black_hole"函数修改一个全局变量来保证解释器不会优化未使用的函数。 防止计算运行时间时不准确

def black_hole(value):

```
global _BLACK_HOLE_SINK

_BLACK_HOLE_SINK = value
```

实验结果与分析

在20000次迭代后 这是我的结果(这里的batch_size和训练时一样,是2048):

Total time for Transformer+GA(批处理和并行优化): 42.785322 seconds

Total time for Transformer(未优化,串行执行): 63.076156 seconds

Total time for GA: 48.705127 seconds

Total time for recursive_Strategy: 47.794147 seconds

未优化的Transformer预测与GA比较时,速度大概是GA的0.77%

但是如果使用了批处理 情况就完全不同了

在数据规模到了一定程度后且离线计算的情况下,使用批处理带来的提升是非常明显的,简单的估计,如果未优化的情况,集成Transformer和GA的模型的运行时间应该大于二者分别运行的和:

48.7+63.1= 111.8 s

但是这里的测试中实现的优化过的集成模型运行时间反而是最短的,证明了我们的优化是很有效的

Transformer 的建模范式冲突与归纳偏置分析讨论

我尝试过将训练集的max_score(即最佳出牌策略的对应得分,也就是此时可能得到的最高分)也加入训练 ,使用A,B作为输入,max_score作为目标 如果单单看预测出的max_score的准确性 结果非常好 ,模型预测的得分与真实值之间的 差值 普遍是极小的

但是经过测试,生成的模型的对于max_score高准确度预测仅仅限制于A,B的长度分别为6和3时,也就是说模型并没有学会"真正"的计算得分的通用方法,比如当length(A)=9而不是6时,预测结果会很糟糕

我也尝试过将max_score额外加入,作为辅助特征用来指导best_move的生成 但遗憾的是几乎没有区别,也就是模型无法意识到max_score和best_move之间的关系 所以我放弃了这个想法,依然只使用A,B作为输入特征 因为这些研究的具体步骤其实和我之前展示的transformer实验过程几乎一样,只是换了几个参数,所以训练细节和评测不再展开详述了

这是一个值得注意的的信号,如果在使用相同架构的Transformer预测max_score的模型并没有真正学会怎么玩这个游戏,那么Transformer预测best_move的模型呢?

置换不变性测试

使用一个完全符合要求的A,B样本, 在其他数据不变的情况生成B的全排列与原样本组合,扩充该样本,使用这个测试集测试transformer模型 测试结果与原样本一致,则说明模型是置换不变的

这是我随机抽取的两个样本的测试结果

Recursive表示递归得到的理论最大得分

A:[11, 13, 3, 10, 12, 6], B: [13, 8, 1], Transformer 结果: 66 Recursive 结果66 A:[11, 13, 3, 10, 12, 6], B: [13, 1, 8], Transformer 结果: 66 Recursive 结果66 A:[11, 13, 3, 10, 12, 6], B: [8, 13, 1], Transformer 结果: 45 Recursive 结果66 A:[11, 13, 3, 10, 12, 6], B: [8, 1, 13], Transformer 结果: 45 Recursive 结果66 A:[11, 13, 3, 10, 12, 6], B: [1, 13, 8], Transformer 结果: 38 Recursive 结果66 A:[11, 13, 3, 10, 12, 6], B: [1, 8, 13], Transformer 结果: 38 Recursive 结果66

A:[6, 5, 9, 11, 10, 13], B: [6, 3, 7], Transformer 结果: 70 Recursive 结果70 A:[6, 5, 9, 11, 10, 13], B: [6, 7, 3], Transformer 结果: 70 Recursive 结果70 A:[6, 5, 9, 11, 10, 13], B: [3, 6, 7], Transformer 结果: 12 Recursive 结果70 A:[6, 5, 9, 11, 10, 13], B: [3, 7, 6], Transformer 结果: 12 Recursive 结果70 A:[6, 5, 9, 11, 10, 13], B: [7, 6, 3], Transformer 结果: 12 Recursive 结果70 A:[6, 5, 9, 11, 10, 13], B: [7, 3, 6], Transformer 结果: 12 Recursive 结果70

对比同一个样本组合中,不同的Transformer的预测结果,和保持不变的真实的理论最大得分,这足以反驳transformer模型是置换不变的

结论

经过对代码的详细审查，可以明确得出结论：该 TransformerMovePredictor 模型对输入序列 B 的元素顺序高度敏感。对于同一组手牌，不同的初始排列顺序将导致模型生成不同的特征输入和最终的策略预测。该模型在设计上并非“置换不变”的（Permutation-Invariant）。

原因剖析

这种顺序敏感性源于数据处理和模型架构两个层面：

特征工程层面的顺序依赖

future_score特征，为每个卡牌元素生成特征向量时，引入了对顺序的明确依赖。

> 关键特征 `future_score_ratio`：该特征的计算基于当前元素在列表 `B` 中的后续所有元素（`remaining_B = B[i + 1:]`）。这意味着，一个元素所关联的“未来得分”特征值，直接取决于它在输入序列中的位置。
> 示例：对于手牌 `B = [10, 20, 30]`，在为元素 `10` 计算特征时，其“未来”手牌是 `[20, 30]`。而对于手牌 `B = [30, 20, 10]`，在为元素 `10` 计算特征时，其“未来”手牌是空集 `[]`。这导致了同一个卡牌元素在不同输入顺序下，其输入到模型中的特征向量截然不同。

模型架构层面的顺序敏感性

Transformer 架构本身是为处理有序序列而设计的，其核心组件强化了对顺序的依赖。

> 位置编码 (`Positional Encoding`)：模型在输入层显式地将每个位置的独有编码信息叠加到特征向量上。因此，即使两个元素的特征向量完全相同，只要它们位于输入序列的不同位置，模型接收到的初始信息就是有区别的。
> 自注意力机制 (`Self-Attention Mechanism`)：该机制通过计算序列中所有元素之间的相互关系来更新每个元素的表示。整个计算过程是全局的，并且依赖于元素的相对和绝对位置。输入顺序的任何变动都会彻底改变注意力得分的分布，从而导致最终输出的差异。

当然可以使用像集成GA一样,求出所有的B的全排列,然后对每个B的排列与A的组合进行预测,然后取预测结果的最大值,来掩盖置换不变的问题 具体的方法之一是TAA(测试时增强) 但是带来的是的length(B)!倍的预测,随之而来的是高得多的运行时间

如果更改模型架构,可以解决顺序敏感性,比如使用set_Transformer的思想,去掉位置编码,使用置换不变的聚合函数，命令模型将 B 视为一个无序的集合来理解 但问题是B是无序的,但是A是有序的,这可能需要混合模型架构 让我们进行一些探索和尝试:

混合模型transformer架构

混合策略Transformer模型相比之前的模型,核心区别在于**将待出牌组（B牌）的处理方式从“有序序列（Sequence）”的视角，转变为“无序集合（Set）”的视角**。使得新模型在泛化能力、健壮性以及解决置换不变性问题上取得了显著的进步。 (实现代码见 code_path)

具体而言，其设计上的主要差别体现在以下两个方面：

1. **架构核心：从序列展平到基于查询的集合处理，实现了置换不变性**

旧版模型将B牌的特征向量按输入顺序展平后送入决策头，这使其输出严重依赖于B牌的输入顺序，未能解决置换不变性问题,并且B牌的输入顺序其实是不应该被考虑的,它应该是无序的

新版架构借鉴了现代目标检测模型（如DETR）的思想，引入了可学习的**“决策槽位查询”（Slot Queries）。这些查询代表了“第一步”、“第二步”等抽象决策意图，通过与各B牌的特征向量进行动态匹配，计算出每张牌与每个决策槽位的适配度。摆脱了对输入顺序的依赖，从根本上解决了置换不变性**问题

2. **训练与解码：从模糊指令到精确对齐，提升了泛化能力与健壮性**

   这一先进架构的优势进一步体现在训练与解码环节，直接转化为更强的泛化能力与健斥性。

   - **在泛化能力上**：由于模型不再关心牌的输入顺序，它被迫学习更本质的规律，即**特定特征组合（如牌值、未来得分潜力等）与出牌时机之间的关联**。这种基于内在属性的知识具有普适性，可以泛化到训练中未见过的全新牌局组合。同时，这种设计也构成了**隐式的数据增强**，进一步强化了模型的泛化表现,future_score的计算不再基于3张牌的特点,使用一个通用、可扩展的动态贪心算法，取代了旧版硬编码、仅限两步的浅层推算

   - **在健壮性上**：首先，新模型的训练过程通过精确的**损失函数对齐**（使用`gather`操作），确保了模型接收到的监督信号是清晰、无矛盾的，避免了学习到虚假关联，使训练过程更稳定。其次，在解码时，模型采用**匈牙利算法**来解析输出，而不是简单的贪心算法,以此确保找到的是全局最优的出牌策略，而非局部最优解。这使得模型的决策过程更为稳健可靠，不易受微小扰动影响。

经过验证,它通过了置换不变性的测试

接下来我们用同样的方式去评测原来的transformer和新的混合模型

```
| 指标                 | 真实值       | original_transformer (算法1) | Hybrid
Transformer (算法2) |
|----------------------|--------------|------------------------------|------------
----------------|
| **平均得分**         | 59.22        | 58.59                        | 46.36
|
| **中位数得分**       | 59.00        | 59.00                        | 47.00
|
| **得分标准差**       | 11.25        | 11.69                        | 14.27
|
| **最低得分**         | 21           | 2                            | 2
|
| **最高得分**         | 98           | 97                           | 96
|
| **均方误差 (MSE)**   | -            | 12.64515                     | 275.2705
|
| **运行时间 (秒)**    | -            | 82.58                        | 76.89
|

---

### 偏差分析

#### original_transformer (算法1)
| 偏差类型             | 偏差值       |
```

```
|---------------------|--------------|
| 均值偏差             | -0.63        |
| 中位数偏差           | 0.00         |
| 标准差偏差           | 3.50         |
| 最低得分偏差         | -52          |
| 最高得分偏差         | 0            |

#### Hybrid Transformer (算法2)
| 偏差类型             | 偏差值       |
|---------------------|--------------|
| 均值偏差             | -12.87       |
| 中位数偏差           | -11.00       |
| 标准差偏差           | 10.48        |
| 最低得分偏差         | -54          |
| 最高得分偏差         | 0            |

---

### 配对 t 检验

#### original_transformer (算法1)
| 统计量               | 值           |
|---------------------|--------------|
| t 统计量             | -25.580      |
| p 值                 | 4.860e-142   |
| 统计显著性           | 显著 (p < 0.05) |

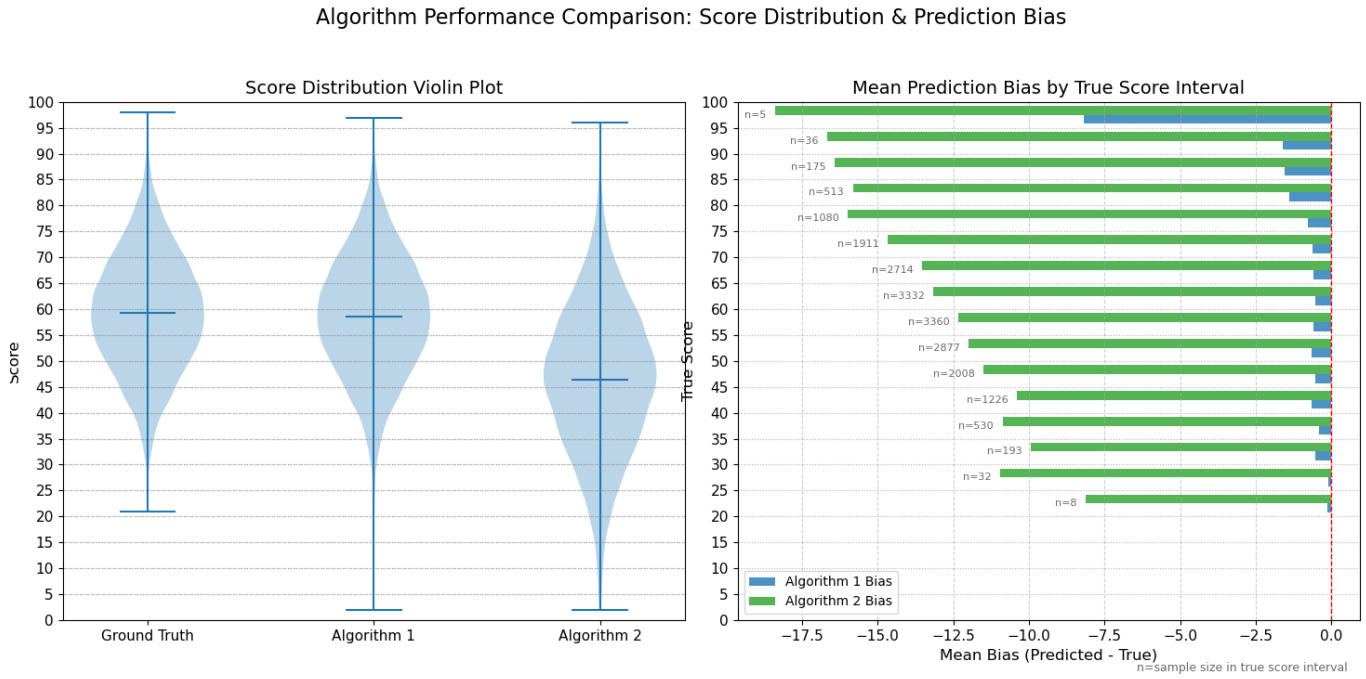
#### Hybrid Transformer (算法2)
| 统计量               | 值           |
|---------------------|--------------|
| t 统计量             | -173.672     |
| p 值                 | 0.000e+00    |
| 统计显著性           | 显著 (p < 0.05) |

---

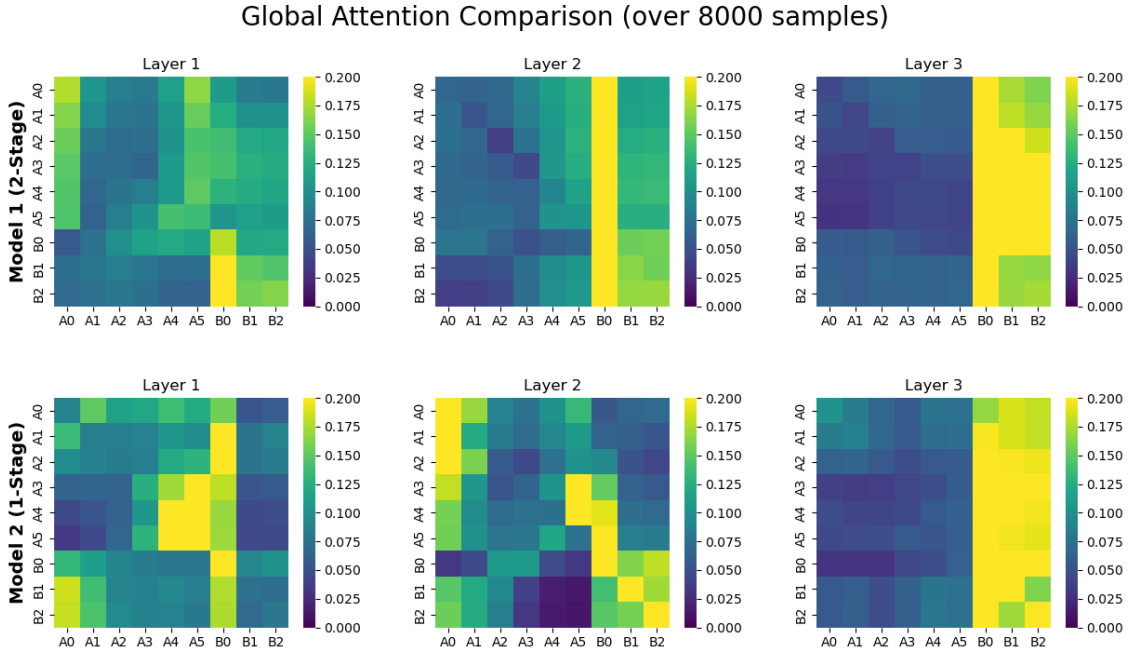
### 效应量分析 (Cohen's d)

#### original_transformer (算法1)
| 指标                 | 值           |
|---------------------|--------------|
| Cohen's d            | -0.181       |
| 影响程度             | 极小         |

#### Hybrid Transformer (算法2)
| 指标                 | 值           |
|---------------------|--------------|
| Cohen's d            | -1.228       |
| 影响程度             | 较大         |
```



可以观察到,虽然模型变得更健壮且通用了(解决了B组的置换不变问题,future_score的计算不再基于"硬编码",而是更通用的动态贪心策略),但是它的准确率下降了,这也在意料之中,比如之前的Transformer的future_score过于理想和完美,而Hybrid Transformer为了适应新的数据,进行的调整让它失去了特例情况的启发式关键优化,为了验证这一点,可以做出是两个transformer的注意力热力图进行分析



从图中可以观察到，**模型1(混合策略的transformer)** 的注意力分布呈现出显著的**逐层收敛（layer-wise convergence）**特性。在浅层网络中，其注意力尚为分散，但随着网络深度增加，注意力权重迅速且高度集中于少数关键信息单元（如图中的 B1 和 B2 ）。从第一阶段（较浅层）:的对所有输入特征（A组和B组）进行初步的、全面的信息提取和编码。到第二阶段（较深层，Layer2,Layer 3）,模型的任务变得专一化。它逐渐停止了对A组内部信息的进一步处理，而是利用已经编码好的A组信息，全力处理B组特征的内部关系，以及A、B两组特征之间的交互关系。这也呼应了混合策略在设计上对A组与B组使用了不同的策略

与此形成鲜明对比的是，**模型2(之前的transform)** 的注意力在深层网络中依旧保持了**较为分散的分布**。尽管对关键单元 B1 和 B2 的关注有所增强，但模型同时在上下文单元之间维持了相当的注意力权重，保留了更多的局部和特异性信息。这揭示了一种不同的学习模式，即单阶段模型倾向于保留更多的上下文细节，从始至终，模型都将所有输入特征（A和B）视为一个整体进行处理。每一层都在全局范围内更新对所有特征的理解，而不会在后期将注意力从某个部分移开。这是标准Transformer编码器的行为方式,这也导致了建模与原问题产生了一定冲突.

第三部分 服务器搭建与游戏架构

游戏中PVE场景,策略是即时性的,而且就算是性能较低的串行实现,AI的计算单次请求的时间足够短了(63/20000=0.00315s=3.15ms),因此我们沿用只使用cpu的串行GA+transformer集成模型,因为它更适合用来构建我们的游戏的本地服务器

服务器会与unity游戏(客户端)进行数据交互,客户端传入A,B 服务器会根据传入的A,B进行预测,返回策略(best_move),然后返回预测结果给客户端进行电脑玩家的自动操作

这里使用的web框架是flask

代码实现见 server.py(code_path)

然后使用docker对server容器化,这样做能将它和它的运行环境打包起来,让它脱离原本的开发环境也能运行 经过测试打包后的server能正常运行

unity项目架构

服务器容器化与使用

选择unity作为构建游戏客户端的工具主要是因为它是一个游戏引擎,提供了游戏开发的便捷,功能相对简单易用,也足够强大.

使用了专门为unity项目设计的 Unity Version Control (UVCS/ plastic scm) 作为版本控制工具

为了方便移植镜像,将编写的server项目使用docker容器化并打包成 gameserver_v1.tar

步骤说明:

在windows系统中下安装docker desktop后

完成项目的requirement.txt文件

用docker init创建dockerfile文件并作必要的修改

编辑.dockerignore,忽略对于服务器项目不需要的文件

在windows系统里的服务器项目根目录下:

构建镜像: docker build -t gameserver:1.0 .

导出镜像 docker save -o ./gameserver_v1.tar gameserver:1.0

基于打包好的镜像启动服务器的方式是:

1.载入tar格式镜像(仅初始部署需要载入一次,镜像搭建后一般不需要再次载入)

""" docker load -i gameserver_v1.tar """

2.依据镜像创建并启动容器 """ docker run -p 5000:5000 --name gameserver gameserver:1.0 """

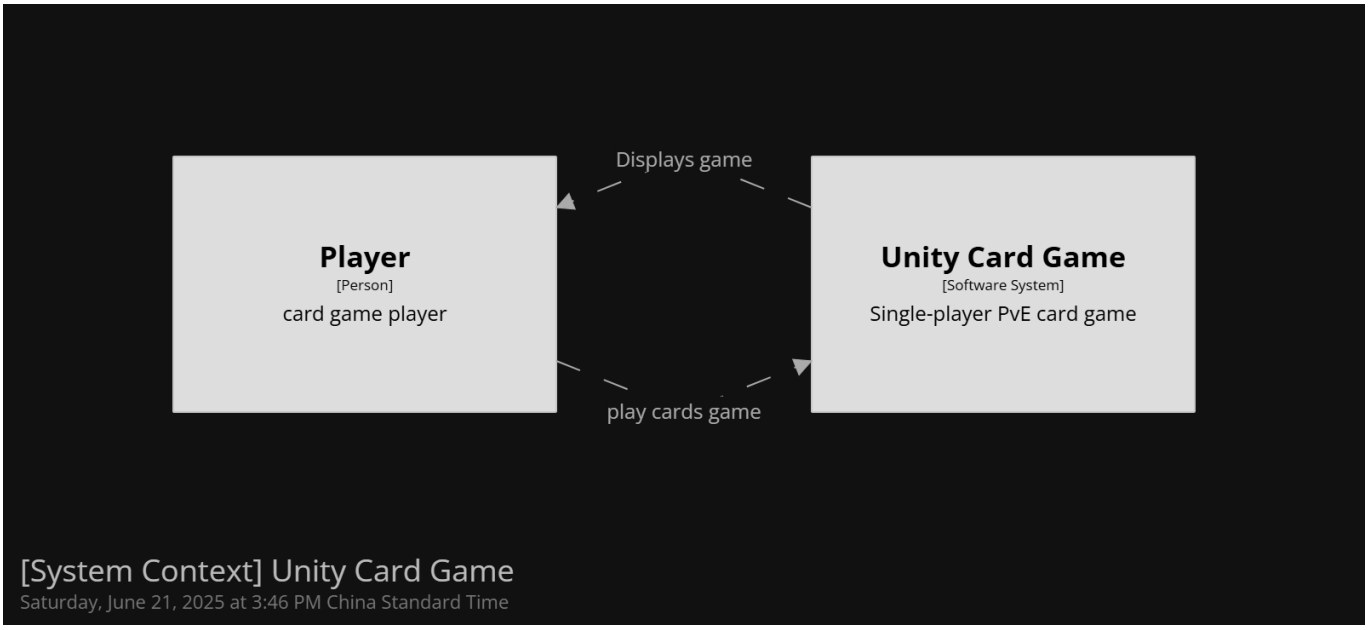
unity前端架构

Unity的编程范式与通用软件开发有显著区别。unity的C#脚本代码与unity本身的组件的深度耦合,要完整理解一段代码的功能，必须首先对unity引擎本身有一定的理解。

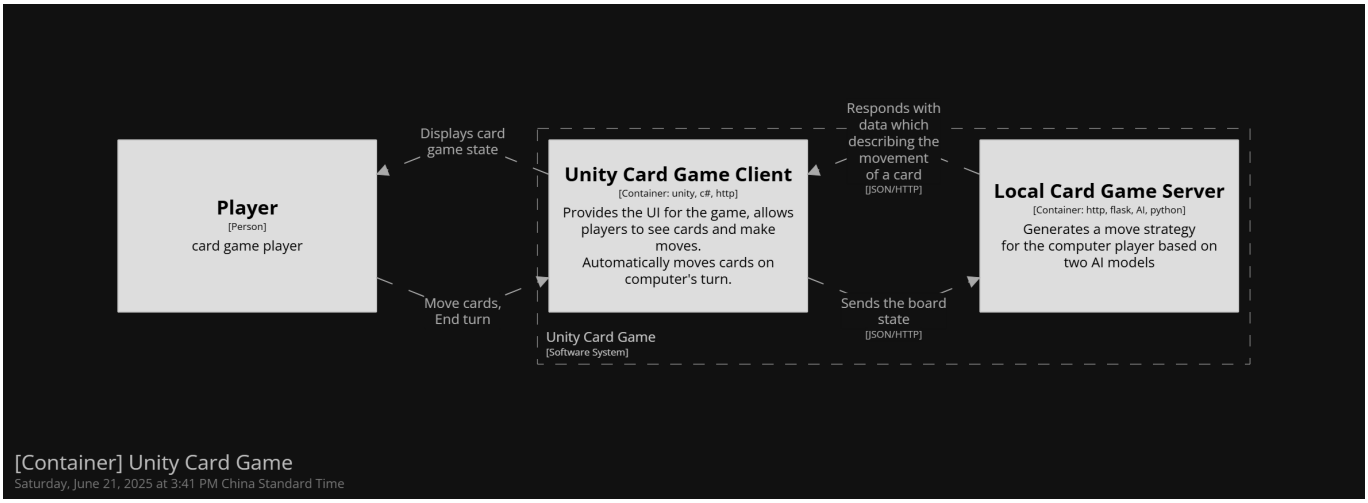
我是基于unity社区框架[2]开发的,其中在代码层面,最难的是 理清楚框架API的逻辑和复杂的继承关系 和 如何使用各种API与unity的组件结合编程,来达成我想要的功能,这很难通过文字描述,并与本文的内容主题有一定脱节.所以在这里我给出了我设计并实现的unity架构的C4 model 图片 来解释这个unity客户端,将我们的关注点放在软件架构上:

使用drawio绘制c4 model草稿后,编写dsl脚本,并渲染为png格式图片:

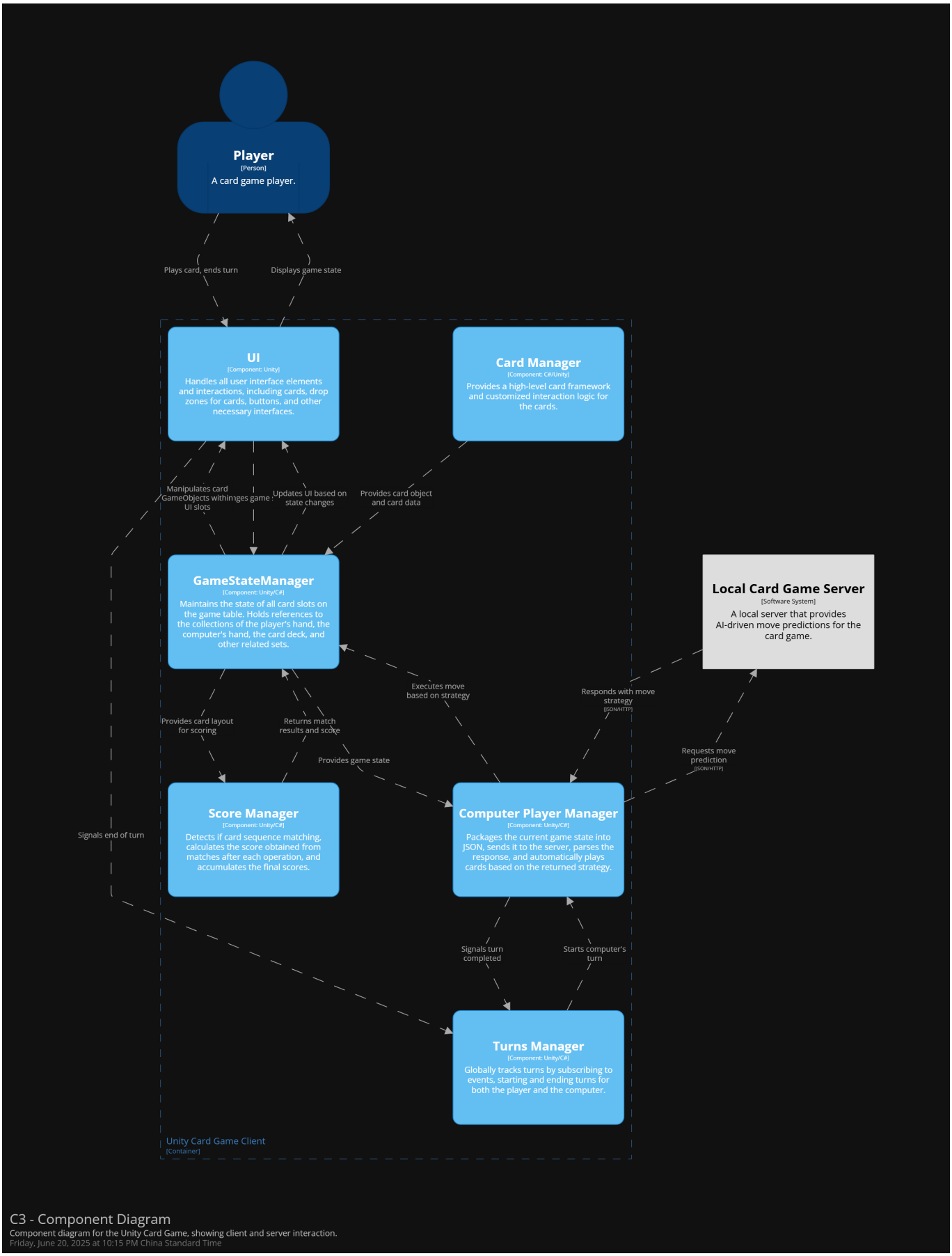
C4 model架构图



Context Diagram



Container Diagram



Component Diagram

这里举例介绍一些这个unity客户端使用的设计模式

单例模式 负责模块功能的类的对象(比如TurnsManager,CardManager等等),它们有且只有一个全局的实例,避免资源重复初始化

适配器模式 在客户端的内部,电脑行动策略是一个数组,但是请求服务器后得到的行动策略是json格式的对象 这里需要通过ServerManager解析json对象进行适配

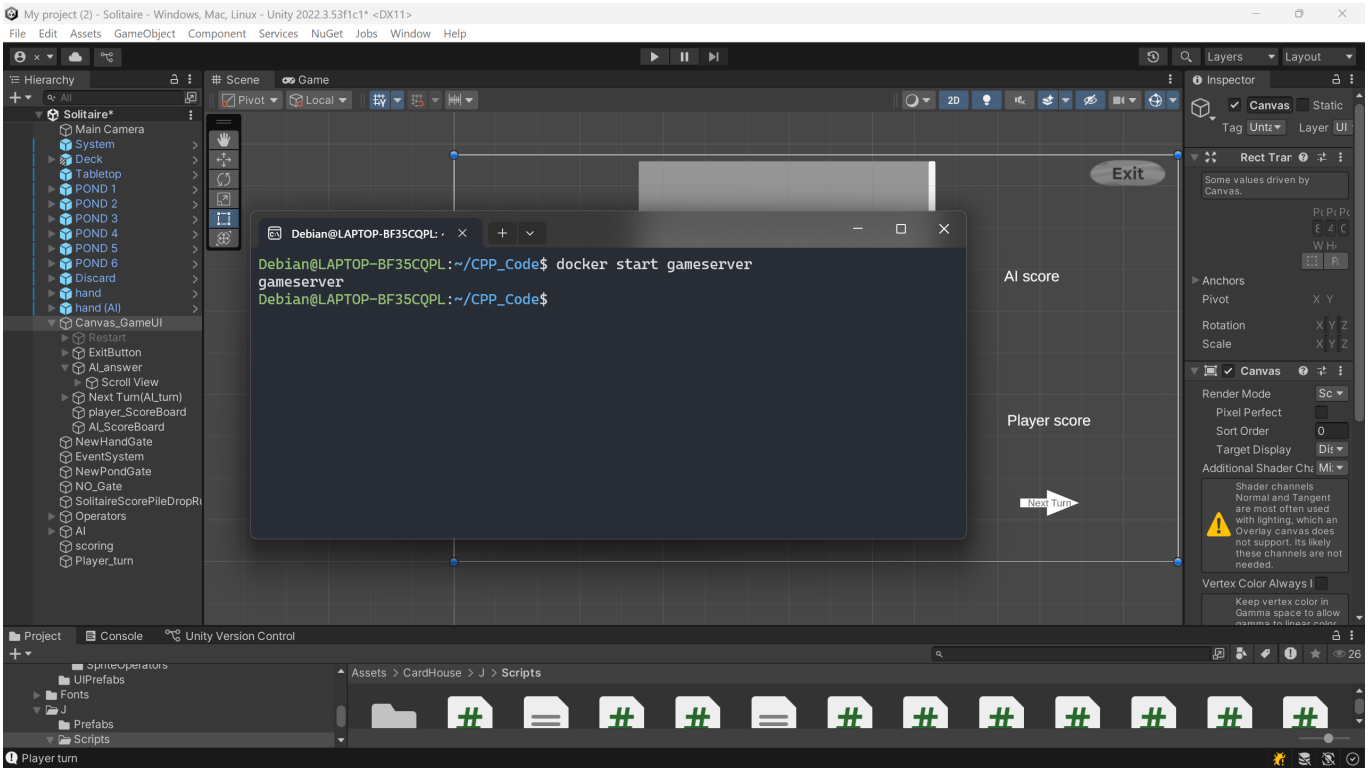
观察者模式 在回合的开始,结束 或者能得分的情况(在我们的项目中叫做匹配)发生后,需要利用发布-订阅机制来发布状态改变信号,通知所有订阅该事件的观察者做出相应的逻辑行为

原型模式 在游戏中,存在着一个可视化,可复用的卡牌预制组件(prefab) 它规定了卡牌的最基本框架(模型、材质、脚本、碰撞箱等等),卡牌游戏场景中的每一个卡牌都是基于这个预制组件进行高效的克隆,再自定义得到的,这个prefab是所有卡牌的原型对象 (The Prototype)

状态模式 在游戏场景里,卡牌的操作逻辑需要根据不同的状态改变

比如 是否是玩家的手牌 (玩家不能移动电脑的手牌) 是否在弃牌堆中 (在弃牌堆的牌不能再操作)

游戏实机演示



我在wsl2上运行的debian 终端输入docker start gameserver来启动服务器



在unity项目的开发界面开始游戏,画面中间的是场上的牌,共6张牌(卡牌游戏最优策略问题中的A) 下方的3张牌是玩家的手牌(卡牌游戏最优策略问题中的B) (上方透明对话框的的文字是我调用Qwen* API在线生成的随机问候,我设定了几个初始人格:

{ "Absent-minded", "Graceful", "Impatient", "Emotionally fragile", "Pessimistic", "Gentle", "Competitive", "Vain", "Shy", "Nervous" ,"relaxed","anxious" } 在本地随机选择一个(使用本地随机而不是让LLM随机的原因是,经过我测试,LLM有对积极人格选择的倾向),作为Prompt 之一,结合其他prompt让LLM生成一段话,向玩家问好,这次随机到的人格是Pessimistic

> [Qwen]* 阿里巴巴开发的LLM



首先是玩家回合 我打出3张牌 注意前两张没有造成匹配,而最后第最后一张点数与场上的牌的第一张点数相同,都是2 所以是将匹配所有卡牌并得分

玩家分数将被更新为所有卡牌的点数之和

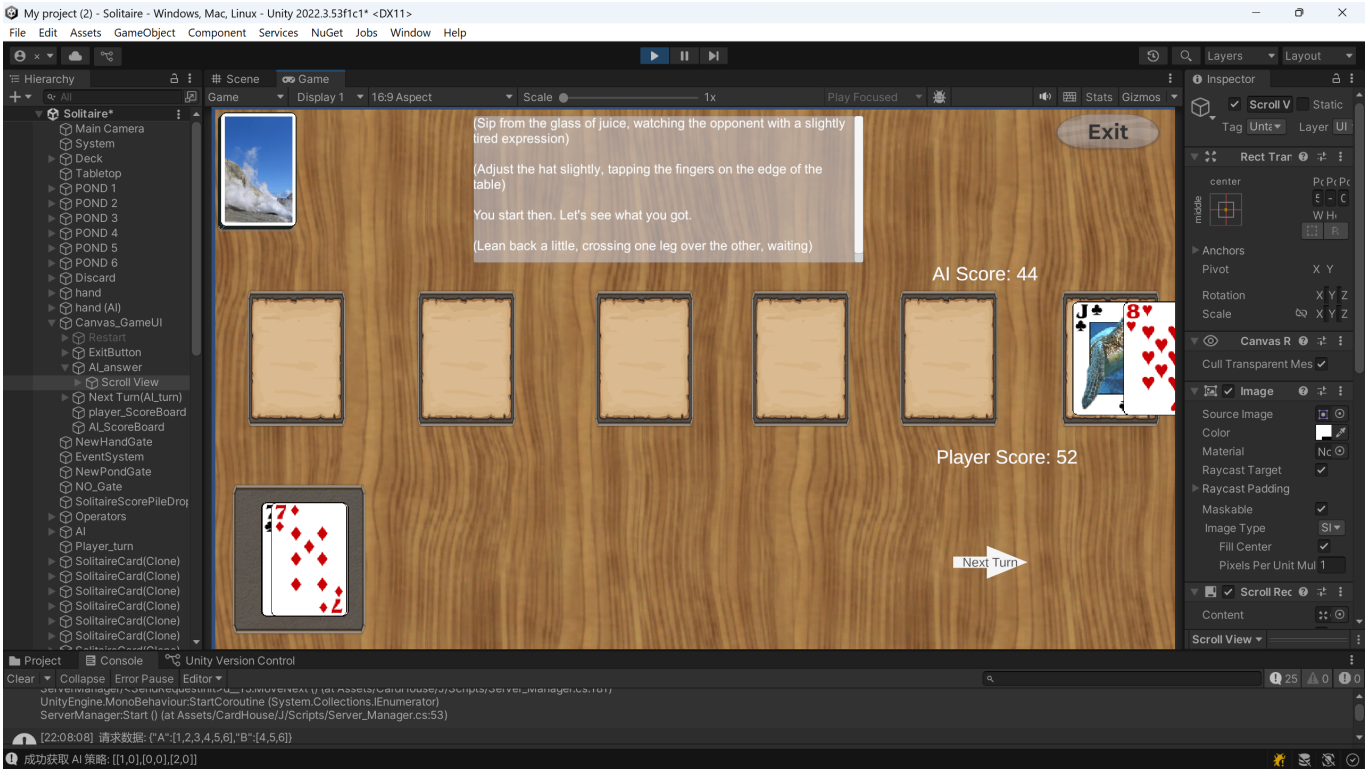
> J,Q,K的点数被视为11,12,13



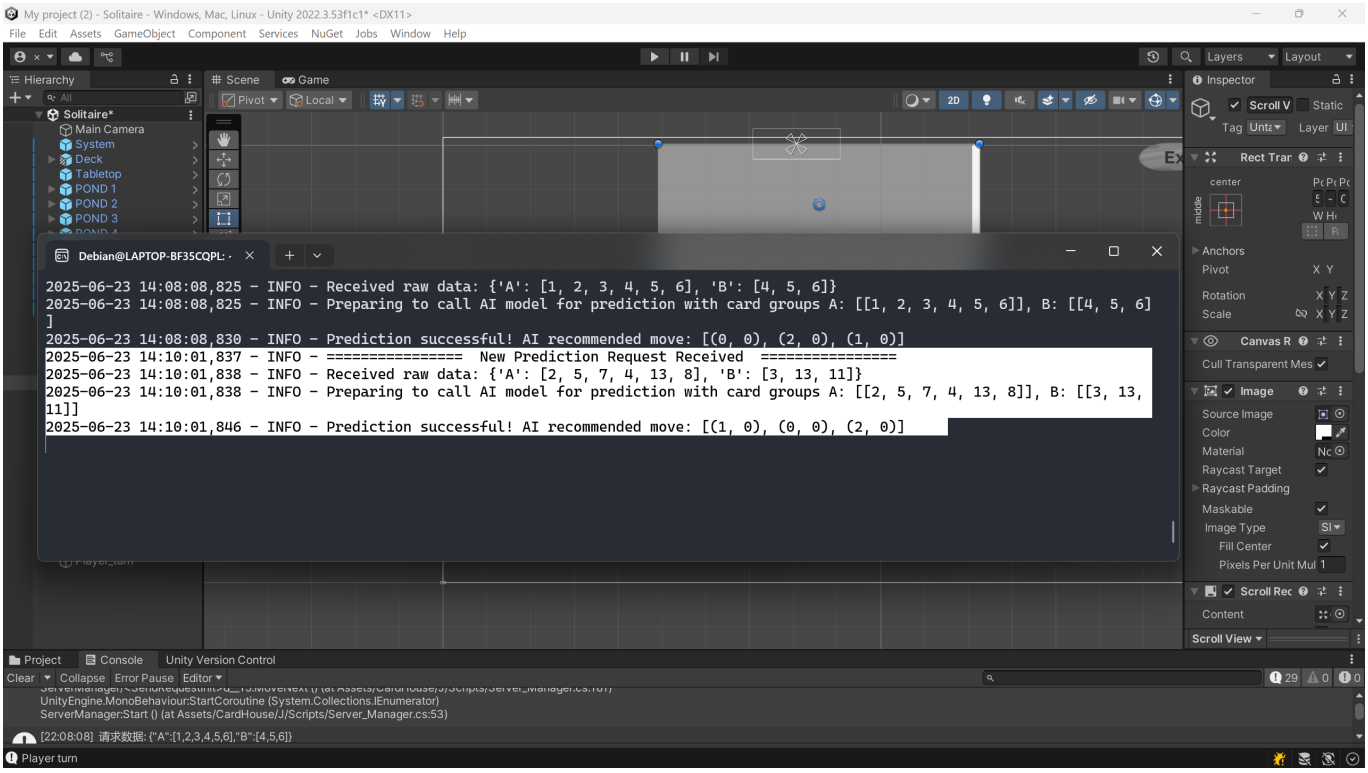
玩家分数被更新为52,玩家结束回合,进入电脑回合 电脑手牌被初始化为K,J,3,场上的牌被初始化为2,5,7,4,K,8

> 这里电脑手牌的排序方式[K,J,3],根据的是电脑收到牌的先后顺序

,电脑将手牌中的K插入场上的牌的最前面.匹配得分,并被消除,场上的牌变为[8],电脑得分44



已经无法得分,但是电脑依据策略将牌插入场上剩余的牌中的最前面



打开服务器日志,可以观察到电脑的实际出牌策略与服务器收发的数据一致(日志高亮部分的上方输出是服务器初始化时固定的通讯检查)

附录:

1.围绕 "出牌策略最大化得分的变体: 是否存在一个出牌策略，使得最终总得分**恰好等于** K 也是NP-hard的" 的讨论

我们同样借助NP-hard问题——**子集和问题 (Subset Sum)**,通用的规约结构,但这次使用karp-reduction来证明这个变体的NP-hard性质。

第一步：定义相关问题

1. **卡牌游戏优化问题 (`CARD_GAME_OPT`)**
    - **实例:** 两个初始卡牌列表 `A` 和 `B`，其中的卡牌点数为正整数。
    - **问题:** 寻找一个出牌策略，使得最终获得的总得分最大化。
2. **卡牌游戏判定问题 (`CARD_GAME_DEC_EQ`)** 为了进行规约，我们定义其"恰好等于"的判定版本：
    - **实例:** 两个初始卡牌列表 `A` 和 `B`，一个目标整数 `K`。
    - **问题:** 是否存在一个出牌策略，使得最终总得分**恰好等于** `K`?
3. **子集和问题 (Subset Sum)** 这是一个经典的NP-complete问题 [1]。
    - **实例:** 一个由 `n` 个互异正整数构成的集合 $S = s_1, s_2, \ldots, s_n$ ，以及一个目标整数 `T`。
    - **问题:** 是否存在 `S` 的一个子集 `S'` $\subseteq$ `S`，使得 `S'` 中所有元素的和恰好等于 `T`？

第二步：构造规约

我们的目标是构造一个算法，该算法能在多项式时间内，将任何一个子集和问题的实例 `(S, T)` 转换成一个等价的卡牌游戏判定问题 `(A, B, K)` 的实例。

给定一个子集和实例 `(S, T)`，我们按如下方式构造 `CARD_GAME_DEC_EQ` 的实例：

1. **选择一个足够大的基数 `X`：** 令 $Sum(S)$ 为集合 `S` 中所有元素的总和。我们选择一个整数 `X`，使其满足 $X > Sum(S)$。由于`T` 在子集和问题中有意义时必然不大于 $Sum(S)$，因此 `X` 也必然大于 `T` 和 `S` 中的任何一个元素。这个 `X` 可以在多项式时间内计算并表示出来（例如，取 $Sum(S) + 1$）。
2. **构造初始列表 `A` 和 `B`:**
    - `A = [ X ]`
    - `B = [ X ] + S` (这里 `+` 表示列表的拼接)
3. **设定目标得分 `K`:**
    - `K = 2X + T`

此转换过程（计算`X`，创建`A`和`B`）只涉及基本的算术和列表操作，其时间和空间复杂度都与输入 `S` 的大小成多项式关系，因此这是一个多项式时间规约。

第三步：证明规约的正确性

现在，我们必须证明：**子集和问题有解，当且仅当（if and only if）我们构造出的卡牌游戏判定问题有解。**

(⇒) 充分性证明

假设子集和问题有解，则卡牌游戏存在策略使得分恰好等于 `K`。

如果子集和问题有解，那么存在一个子集 $S' \subseteq S$，其元素之和 $Sum(S') = T$。我们可以构造如下策略：

1. 玩家通过一系列操作，将 `S'` 中的所有卡牌排列在两个 `X` 卡牌之间，形成 `[X, S', X]` 的得分结构。
2. 根据游戏的匹配规则（如证明草稿中所述），这个结构会产生 $2X + Sum(S')$ 的得分。
3. 由于 $Sum(S') = T$，所以这一步的得分为 $2X + T = K$。
4. 在这次得分后，两个 `X` 卡牌和 `S'` 中的所有卡牌都被移除。场上剩下的卡牌是集合 $S \setminus S'$。因为原始集合 `S` 的元素是互异的，所以剩下的这些卡牌点数也都是互异的，无法再形成任何匹配得分。
5. 因此，最终总得分**恰好**是 `K`。

(⇐) 必要性证明

假设卡牌游戏存在策略使得分恰好等于 `K`，则子集和问题有解。

如果存在一个策略使得最终得分恰好为 $K = 2X + T$，我们来分析得分的来源。在这个构造的实例中，唯一能形成匹配的卡牌只有两张 `X`。

1. **情况一：两张 `X` 直接相邻匹配。** 如果策略使得两个 `X` 直接相邻形成 `[X, X]` 结构，那么得分将是 `2X`。之后，场上只剩下集合 `S` 中的互异卡牌，无法再得分。最终总分是 `2X`。但这不等于$K = 2X + T$ (因为 `T` 是正整数)。所以，这种策略无法得到目标分数 `K`。

2. **情况二：两张 `X` 不相邻匹配。** 要使两个 `X` 匹配，它们之间必须夹着一些来自 `S` 的卡牌。设这些被夹在中间的卡牌构成的子集为 $S_{mid} \subseteq S$。此时形成的得分结构为`[X, S_mid, X]`。

   - 该结构的得分是 $2X + Sum(S_{mid})$。
   - 得分后，场上剩下的卡牌无法再次匹配。因此，这是最终总得分。
   - 为了使总得分恰好等于 `K`，必须有：$2X + Sum(S_{mid}) = K = 2X + T$。
   - 这直接导出 $Sum(S_{mid}) = T$。

   这意味着我们找到了 `S` 的一个子集 (S_{mid}) 其元素之和等于 `T`。因此，子集和问题有解。

我们已经证明了两个方向的等价性。由于 `Subset Sum` 是 NP-complete 的，并且我们构造了一个到 `CARD_GAME_DEC_EQ` 的多项式时间规约，我们得出结论：

结论i: 卡牌游戏的判定变体问题 (`CARD_GAME_DEC_EQ`) 是 NP-hard 的。

关于优化问题、判定问题及其变体的关系

现在我们探讨三个问题的内在联系：

> 判定问题 (Decision Problem): 即与优化问题直接对应的，“是否存在一个策略，使得总得分不小于一个给定值K？” (∃ strategy_score ≥ K)。
>
> 优化问题 (Optimization Problem): 即项目核心所研究的“最大化游戏总得分”。(已证明为NP-hard)
>
> 变体判定问题 (Variant Decision Problem): 上文已经证明的，“是否存在一个策略，使得总得分恰好等于一个给定值K？” (已证明为NP-hard)

这三者之间存在着紧密的计算复杂性联系，澄清这种关系有助于我们更深刻地理解原问题的计算难度。

优化问题与其判定问题的等价性 (Turing Equivalence)

“得分最大化”的优化问题与其直接对应的“∃ strategy_score ≥ K ?”判定问题，在多项式时间内是图灵等价的。这意味着，如果我们拥有一个能解决其中任一问题的“神谕机”（Oracle），我们就能在多项式时间内解决另一个。

> （优化 ⇒ 判定）： 若能解决优化问题，则可以得到最大分 max_score,
>
> 比较 max_score 与 K 的大小，
>
> 如果 max_score>=K,那么由于max_score就是一个strategy_score,故存在策略strategy_score ≥ K,所以判定结果为真
>
> 如果 max_score<K,那么∀strategy_score<=max_score<K,得到 ∀strategy_score<K 所以判定结果为假
>
> 即可在常数时间内回答(∃ strategy_score ≥ K ?)。
>
> （判定 ⇒ 优化）： 若能解决“score ≥ K？”的判定问题，我们可以通过在所有可能得分范围内进行二分搜索，时间复杂度为O(log n),在对数次调用神谕机后，精确地找到 max_score。

结论ii：从计算难度的角度看，这两个问题是等价的。证明“score ≥ K”判定问题的NP-Hard性，就等同于证明了原优化问题的NP-Hard性。

所以,

> 优化问题是 NP-Hard 的（见正文分析）。
>
> 优化问题对应的判定问题是 NP-Hard 的（结论ii）。
>
> 判定问题的变体是 NP-Hard 的（结论ii）。
>
> 我们证明了“score = K”这一变体判定问题是NP-hard的。这揭示了游戏规则内在的计算困难度。—即便是精确地达成一个特定分数，也已是理论上的难题。

“score = K”问题是“score ≥ K”问题的一个特例。任何一个能够解决“score ≥ K”这一更一般性问题的算法，都必须能够正确处理其所有特例。因此，解决“score ≥ K”问题在最坏情况下的时间复杂度，必然不低于解决“score = K”问题的复杂度。由此可得，“score ≥ K”这一直接对应的判定问题同样是NP-Hard的。

> 由于优化问题与“score ≥ K”判定问题在计算上等价，我们最终为原项目所研究的“得分最大化”问题难以在多项式时间内精确求解，提供了坚实而完备的理论支撑。

故我们的逻辑链条为 变体判定问题是NP-hard的 -> 优化问题对应的判定问题是 NP-Hard 的 -> 优化问题是 NP-Hard 的

这些结论共同指向一个核心事实：该卡牌游戏的核心策略的多种形式的提出，其计算难度都触及了NP问题的理论上限。因此，在P≠NP的假设下，不存在解决此问题的高效（多项式时间）算法。

这为我们项目中的核心决策——即放弃寻找通用精确解，转而采用AI代理模型来高效地逼近最优解——提供了多重的,更坚实的理论依据

2.特殊的树结构:

在思考优化方法时,我尝试了使用树来直观的的表示这个问题,以此得到更好的算法, 期间我发现了这样的树结构

一棵深度为k的树，根节点值为0，所有叶子在第k层。每个非根节点的值都是1到n之间的正整数。对于任意从根到叶子的路径，节点值严格递增。这棵树包含且仅包含所有满足上述条件的路径 问这棵树有多少个叶子节点

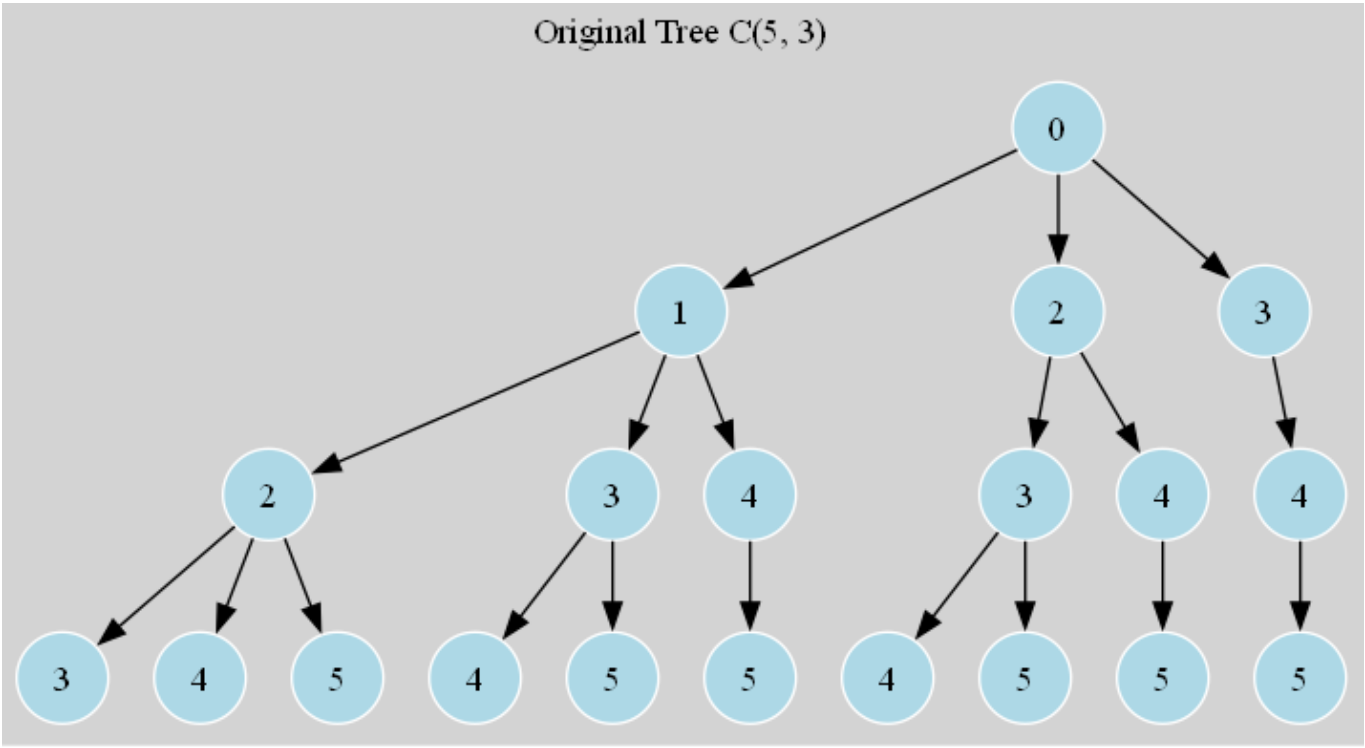
每一条根节点到叶子的路径都需要有一个数字,这相当于 P(n k) 递增这一条件又严格规定了在数字确定后,路径上的顺序有且仅有一种,不可以排列 ,所以结果是 P(n k)/k! 也就是C(n k) 而且每一条路径都直观的展示了组合数的每一种情况

思考在算法分析讨论过的多重集问题{A,A, A,A, A,A, B,B,B}构成的集合中不放回的拿出元素,全部放到一个一维数组里,能构成多少种长度为9的不同的数组 ,这个问题的解是C(9,3)*C(6,6)

为了方便展示 我们缩小规模来观察 假设多重集是{A,A,A,B,B}

这对应了k=3,n=5时 ,将这些元素设定序号[1~5]后,

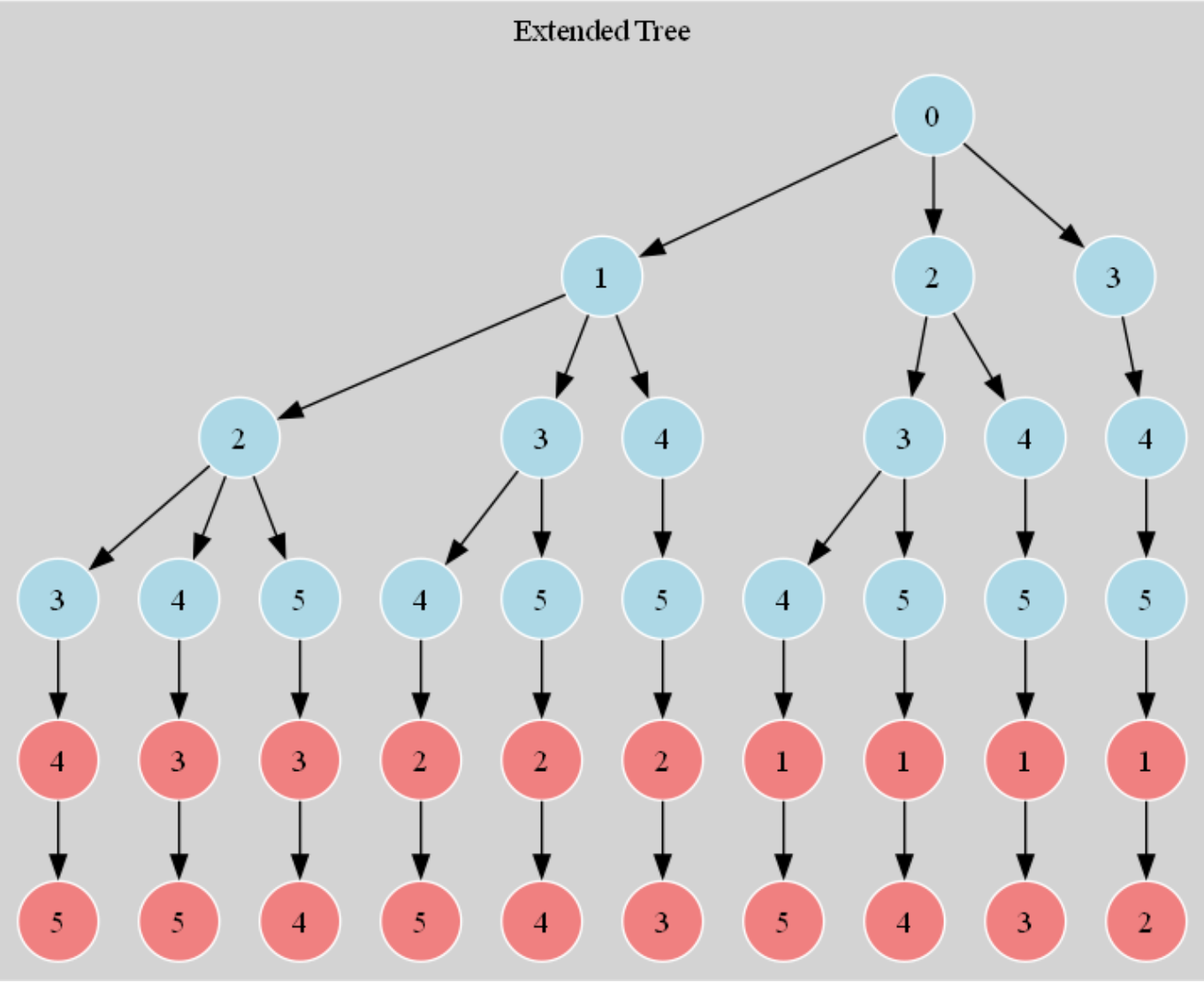
可生成树:



对于每一条路径 将剩下未被选中的(n-k)个元素组成的升序链表连接到树叶上(升序可以扩展为任意的双射规则)

根据C(n, k) = C(n, n-k) 每条根到叶路径(忽略根的值0)都对应了一个不重复的数组可能,并且每条路径的值组成的数组集合一一对应了问题中的每一种数组

依据树的规则,可以将上图扩展为



当然这样的目前的树结构和我们研究的卡牌插入得分最大化问题还有一定差别 还包括将节点再加入(1~13)的"卡牌点数",区分A和B的"类型值",接着加入匹配规则操作(考虑匹配规则后会很复杂,需要为这棵树设计很多操作函数,为了达成这些目标,树的结构可能需要产生巨大改变)等等,但由于篇幅,不再进一步讨论

换一种方式观察这棵树,而且因为这个树的根有n-k+1个子节点 (因为根到叶子的路径长需要为k,且节点上的数字<=n,所以树的子节点的值必须<=n-k+1,否则无法构成长度为k的路径)

考虑每一个根的子节点,

当值为1时,由它到叶子的路径上的节点还有(n-1)种选择,需要再选取(k-1)种 所以有C(n-1,k-1)种组合,也就是这个节点是的C(n-1,k-1)个节点的祖先节点 当值为2时,由它到叶子的路径上的节点还有(n-2)种选择,需要再选取(k-2)种 所以有C(n-2,k-1)种组合,.....

值为(n-k+1)的子节点：还需从{n-k+2,...,n}中选(k-1)个数，有C(k-1,k-1)=1种路径

$$\binom{n-1}{k-1}+\binom{n-2}{k-1}+\cdots+\binom{n-k+1}{k-1}$$

或者

$$\sum_{i=1}^{n-k+1}\binom{n-i}{k-1}$$

而这个和会等于总叶子数,所以

$$\sum_{i=1}^{n-k+1} \binom{n-i}{k-1} = \binom{n}{k}$$

这也就是hockey-stick恒等式 这棵树巧妙的连接了数据结构和组合问题

项目展望与结语

思考文中讨论的范式的泛化可行性,之前提及的Transformer 的建模范式冲突的是不可忽视的问题,并且在更高的数据规模下,目前的Transformer与GA的特征可能都很难适配,比如future_score这一在当前特例的重要特征是基于A,B的长度都很小的情况定制的,它默认了B的长度<=3.此外,获取很高的数据规模的大量训练集可能很困难

当A,B的长度很大时,如果沿用future_score,因为它只提供了length(B)<3后的得分前瞻,这部分前瞻性对整体得分的影响将会很小

之前讨论过的混合策略Transformer由于追求泛化性,和健壮性,失去了对特例的优化,导致性能欠佳,并且本文实现方法基于监督学习,它依然受限于穷举算法得到,或者其他途径收集得到的训练集,以及人工设定的特征;为了实现更通用的AI模型,可以考虑加入强化学习的机制,让AI实现自动的深度推演和规划，做出最终决策.

或者我们还可以把眼光放在client,进行再一步的混合 . 在不使用递归穷举这种时间复杂度的情况下,可以在客户端再加入一个简单的自定义贪心策略的算法.因为虽然AI算法的前瞻性(比如future_score)和transformer捕捉元素间关联的能力很好的提高了mse,但是它其实是与贪心这一策略冲突的,在某些情况下,它反而会因为过于保守而拉低了特例的最低得分,我们可以利用这一点来设计算法,混合服务器策略,贪心算法的代价极小,我觉得这是一个在工程上有价值的尝试

不过想要很好的利用ai代理模型这种启发式方法来处理一般化的NP问题,对硬件和知识的要求极高,非我目前可以完成的

这是我的AI服务器端github项目链接:...

由于unity版本控制系统是邀请制的私有系统,所以我无法直接公开它来让所有人都可以访问我的unity 客户端项目源文件.它的架构与实机演示截图已经在正文部分中详细描述.

引用资料

---

[1] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[2] unity社区卡牌游戏框架

[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. "Attention is All You Need." In *Advances in Neural Information Processing Systems* (NeurIPS), 2017.

```
Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with
neural networks. In Advances in neural information processing systems (NIPS 2014).

Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural machine translation by jointly
```

learning to align and translate. arXiv preprint arXiv:1409.0473. (Published at ICLR 2015)

[4] Holland, J. H. (1975). Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence. University of Michigan Press.

Goldberg, D. E. (1989). Genetic algorithms in search, optimization, and machine learning. Addison-Wesley.

[5] Tanese, R. (1989). Distributed genetic algorithms. In Proceedings of the 3rd International Conference on Genetic Algorithms (pp. 434-439).

Mühlenbein, H., Schomisch, M., & Born, J. (1991). The parallel genetic algorithm as function optimizer. Parallel Computing, 17(6-7), 619-632.

由于项目涉及的技术广,工程量大 我使用了gemini,claude,chatgpt,Qwen等LLM模型进行必要的信息咨询以及部分代码的框架生成与优化